

SMP Integration — Implementation Runbook

Step-by-step lifting procedures and invariants

Implementation Runbook: Executing the SMP → xpack-development Integration

- From scratch — the ordered procedure
- File Manifest — what each step integrates (and the corrections applied)
- Part 0 — Dissolve the devices Repository (do this first)
- 0. Model of Operation
- 1. Prerequisites & One-Time Setup
- 2. The Automation Scripts
- 3. The Verification Gate (reference)
- 4. Part A — Uniprocessor Fixes (Steps 1–13)
- 5. Part B — SMP Infrastructure (Steps 14–23)
- 6. Part C — Releases, New Targets, Add-Only Tests (Steps 24–28)

scripts/smp/release-port.sh "" (local only)

- 7. Part D — Docs & Final Merge (Steps 29–30)
- 8. Quick Reference — Full Loop for Any Step
- 9. Step / Repo / Gate Matrix
- 10. End-to-End Command Flow (From an Empty Folder)
- 11. Doing This For Real — The Practical Playbook

Implementation Runbook: Executing the SMP → `xpack-development` Integration

Companion to `SMP-UPSTREAM-INTEGRATION-PLAN.md` and `MICRO-OS-PLUS-SMP-VS-SINGLECORE-ANALYSIS.md`.

Purpose. The two companion documents explain *what* changes and *why*. This document is the operator's runbook: *how* to actually perform the 30-step integration — the exact branch commands, the file/line edits, the code to paste, the command that tests each step, the gate that decides pass/fail, and the command that advances to the next step. It also ships the helper scripts that automate the repetitive parts so a step is `new → edit → verify → advance`.

Direction of travel: the `smp` branch is the **source of truth** (it already contains every fix). Each step re-creates one cohesive chunk of that work on top of `origin/xpack-development`, verifies it against the 72-test gate, and only then moves on. Nothing is bulk-copied.

Repo model. Every component lives once on `github.com/dan-maio` with a single `origin` remote. The single-core baseline and the SMP source are two **branches** of that same repo: `xpack-development` (integrate into) and `smp` (lift from). So `$UPSTREAM = origin/xpack-development` and the source is `origin/smp` — no second remote.

The helper scripts in §2 exist for real under `../scripts/smp/`; this document is their reference.

From scratch — the ordered procedure

This is the whole job end to end: from nothing to a local `xpack-development` that carries the SMP work, verified, **never pushed**. Each line is the action and the command; the deep detail is in the section named. `smp` is the source of truth and is **never modified**; all commits land on the `step/NN` branches, which descend from `xpack-development`, and are merged onto `xpack-development` only at the end (Step 30), locally.

Phase 0 — set up the workspace (once). 1. **Check the toolchains.** `scripts/smp/check-env.sh` — xpm, cmake, ninja, the pinned GCC/Clang/arm-none-eabi/aarch64-none-elf/qemu. (§1.1) 2. **Clone the repos and fetch both branches.** `WORK=... bash ../bootstrap.sh` clones the core three (kernel + posix-arch + cortexm); also clone the arch repos (`aarch32`, `aarch64`) and `devices` beside them. Each repo has the two branches `xpack-development` (integrate into) and `smp` (lift from). (§1, §10.5) 3. **Register the port dev-links.** In each port: `xpm install && xpm link`. (§2.3, §2.6) 4. **Part 0 — dissolve devices** (once, before Step 1): `scripts/smp/migrate-devices.sh` folds the `devices` SoC/driver code into the arch repos (history-preserving `git subtree`), so the integration proceeds with the six repos upstream already has. (**Part 0**)

Phase 1 — lift the work, one bisectable step at a time (Steps 1–31). 5. For each step `NN`, the loop is **four commands** (§0.3): - `scripts/smp/new-step.sh NN` — fork `step/NN` from `step/NN-1` in the repos it touches. - **apply the step's chunk** — `scripts/smp/chunks/stepNN.sh` (or `part-b.sh` for 14–23), which `git checkout origin/smp -- <files>` and applies the asserted corrections. The exact files + corrections per step are the **File Manifest** below. - `scripts/smp/verify-step.sh NN` — the gate: pristine check + undef invariant (14–23) + link + the 72-test suite. (§2.4, §3) - `scripts/smp/advance-step.sh NN` — commit the touched repos, tag `step-NN-green`. - Or drive the whole range at once: `scripts/smp/run-loop.sh 1 31`. The step groups: **1–13** Part A (uniprocessor fixes), **14–23** Part B (SMP infrastructure), then Part C — **24** port releases, **25** new cores/boards, **26** modular CMake, **27** new test sources (runs `absorb-test-smpl.sh` to dissolve `test_smpl/` → `tests/smp-support/`), **28** add-only platforms (runs `integrate-aarch-harness.sh` to wire the AArch32/64 ports into the standard xpm test chain), **29** docs, **31** `cortexm-pico2`.

Phase 2 — prove it beyond the per-step gate. 6. **Full-matrix gate (MANDATORY):** `scripts/smp/full-verify.sh` — a real `xpm install → link-deps → prepare → build → test` for every platform, one GCC + one Clang. (§3.1) 7. **New-architecture harness suites:** build+run via the framework's own chain, e.g. `cd tests && xpm run test-mutex-stress-qemu --config aarch64-rpi3b-cmake-gcc-debug` (configure → build → `ctest` → `run-qemu.sh`). (Step 28 note) 8. **Optional — newer host compilers:** `scripts/smp/newer-toolchains.sh`. (§3.2)

Phase 3 — land it (local only). 9. **Step 30 — final merge, local:** `scripts/smp/finalize.sh` fast-forwards `xpack-development` to the green `step/NN` chain, drops the dev tooling, and re-runs the gate. **Nothing is pushed.** (Step 30) 10. **The PR is manual.** A human opens/pushes it when ready — the automation never pushes (§2.6). Until then every commit lives only in the local repos.

Minimal re-run from a fresh clone (everything already understood): `check-env.sh` → `bootstrap.sh` (+ clone arch/devices) → `migrate-devices.sh` → `run-loop.sh 1 31` → `full-verify.sh` → `finalize.sh`. No push at any point.

File Manifest — what each step integrates (and the corrections applied)

This is the index of the whole integration: for every step, the exact **files / code chunks lifted from** `origin/smp`, and the **corrections** the step applies while lifting them (single-core projection, stripping later-step content, surgical inserts, destination-clang `-Weverything` hardening, and the few places the baseline is deliberately kept). It is generated from, and must stay in step with, the chunk recipes under `./scripts/smp/chunks/` — each `stepNN.sh` / `part-b.sh` is the executable form of one row here.

Repos: **K** = `micro-os-plus-iii` (kernel), **C** = `micro-os-plus-iii-cortexm`, **P** = `micro-os-plus-iii-posix-arch`. Paths are repo-relative. "whole" = the `origin/smp` file copied verbatim; "projection" = `sc-project.py` strips the `#if defined(OS_USE_SMP_SCHEDULER)` / `NCPU > 1` blocks; "surgical" = only the named hunk is lifted into the baseline file.

Part A — single-core correctness (Steps 1–13, kernel-only unless noted)

Step	Repo	Files lifted from origin/smp	Corrections applied
1	K	<code>include/cmsis-plus/posix/dirent.h</code> , <code>include/cmsis-plus/utils/lists.h</code> (whole)	<code>src/rtos/os-thread.cpp</code> : surgical — only the exported-suspend symbol hunk. Baseline kept for <code>src/libc/stdlib/timegm.c</code> (smp hunk regresses glibc ≥ 2.44 <code>-Werror=missing-prototypes</code>).
2	K	<code>include/cmsis-plus/rtos/os-memory.h</code> , <code>include/cmsis-plus/memory/first-fit-top.h</code> , <code>src/rtos/os-memory.cpp</code> , <code>src/memory/first-fit-top.cpp</code> , <code>src/memory/lifo.cpp</code> , <code>src/memory/block-pool.cpp</code> , <code>src/libc/stdlib/malloc.cpp</code> (whole)	<code>first-fit-top.cpp</code> : diagnostic-pragma wrap. Seven files must land together (else undefined-ref on <code>memory_resource::do_usable_size</code>).
3	K	<code>src/libcpp/new.cpp</code> , <code>src/libcpp/system-error.cpp</code> , <code>src/libcpp/chrono.cpp</code> (whole)	<code>system-error.cpp</code> : add clang <code>-Wexit-time-destructors</code> suppression for the two Meyers-singleton error categories.
4	K	<code>src/rtos/os-c-wrapper.cpp</code> (whole)	Strip Step-9 (<code>os_thread_state_destroying</code> static_assert) and Step-20 (<code>OS_USE_SMP_SCHEDULER</code> CMSIS-v1 affinity) content — Step 4 carries only its own concern.
5	K	<code>src/posix-io/file-descriptors-manager.cpp</code> , <code>include/cmsis-plus/posix-io/file-system.h</code> , <code>include/cmsis-plus/posix-io/net-stack.h</code> , <code>src/posix-io/block-device.cpp</code> (whole)	<code>fd-manager.cpp</code> : file-level clang <code>-Wunsafe-buffer-usage</code> suppression for the new bounds-checked descriptor-array accesses.
6	K	<code>include/cmsis-plus/arm/semihosting.h</code> , <code>src/startup/exception-handlers.c</code> , <code>src/semihosting/c-syscalls-semihosting.cpp</code> (whole)	— (all three clean single-step).

Step	Repo	Files lifted from <code>origin/smp</code>	Corrections applied
7	K	<code>src/rtos/internal/os-lists.cpp</code> , <code>src/rtos/os-timer.cpp</code> (whole)	<code>include/cmsis-plus/rtos/os-thread.h</code> : surgical — only the handler-mode <code>errno-scratch</code> hunk (keep Step-9 <code>destroying=7</code> and Step-20 affinity out).
8	K	<code>src/rtos/os-mutex.cpp</code> (whole)	— (clean single-step).
9	K	<code>include/cmsis-plus/rtos/os-thread.h</code> , <code>src/rtos/os-thread.cpp</code> , <code>src/rtos/os-idle.cpp</code> (projection)	<code>include/cmsis-plus/rtos/os-c-decls.h</code> : surgical — lift the <code>destroying</code> enum but keep Step-10's <code>void* clock</code> field out. (~37 SMP hunks make hand-stripping infeasible → projection.)
10	K	<code>src/rtos/os-condvar.cpp</code> , <code>include/cmsis-plus/rtos/os-condvar.h</code> , <code>include/cmsis-plus/diag/instrumentation.h</code> (whole)	<code>include/cmsis-plus/rtos/os-c-decls.h</code> : surgical — uncomment only <code>os_condvar_t</code> 's <code>void* clock</code> (keep Step-16 SMP klock out).
11	K	<code>src/libcpp/thread-cpp.h</code> , <code>include/cmsis-plus/estd/thread_internal.h</code> (whole)	— (both clean single-step).
12	K	<code>src/rtos/os-mqueue.cpp</code> (whole)	— (clean single-step).
13	K, C, P	K: <code>include/cmsis-plus/rtos/os-decls.h</code> , <code>src/rtos/os-clocks.cpp</code> (whole). C: <code>include/cmsis-plus/rtos/port/os-inlines.h</code> (projection).	First cross-repo step. P: <code>include/cmsis-plus/rtos/port/os-inlines.h</code> — surgical insert of only the <code>clock_highres</code> block (<code>CLOCK_MONOTONIC</code>); the baseline file mixes later-step per-CPU content so neither whole-copy nor projection is safe.

Part B — SMP infrastructure (Steps 14–23, one interwoven blob)

Steps 14–23 are too entangled to lift file-by-file, so `part-b.sh` lands them as one cohesive set across all three repos, then applies the hardening fixes A–J.

Repo	Files lifted from <code>origin/smp</code> (whole, SMP versions)
K	<code>include/cmsis-plus/rtos/os-c-decls.h</code> , <code>os-sched.h</code> , <code>os-thread.h</code> ; <code>src/rtos/os-c-wrapper.cpp</code> , <code>os-core.cpp</code> , <code>os-idle.cpp</code> , <code>os-main.cpp</code> , <code>os-thread.cpp</code>
C	<code>include/cmsis-plus/rtos/port/os-c-decls.h</code> , <code>os-decls.h</code> , <code>os-inlines.h</code> ; <code>src/rtos/os-core.cpp</code>
P	<code>include/cmsis-plus/rtos/port/os-c-decls.h</code> , <code>os-decls.h</code> , <code>os-inlines.h</code> ; <code>include/host_cpu.hpp</code> , <code>include/exception_handler.hpp</code> , <code>include/hw_result.hpp</code> ; <code>src/host_cpu.cpp</code> , <code>src/free-store.cpp</code> , <code>src/board-contract.cpp</code> , <code>src/exception_handler.cpp</code> , <code>src/rtos/os-core.cpp</code>

Corrections applied in Part B (fixes A–J): - **A/B** posix-arch `os-inlines.h`: `struct timespec tp;` → `timespec tp;` (baseline gate is `-Werror=redundant-tags`). - **C** posix-arch CMake: register the new SMP sources **additively** in the port's INTERFACE target (do **not** apply smp's standalone `UOS_SMP_DIR` / `add_subdirectory` CMake); `board-contract.cpp` is board-specific (`#error`s without `PORT_GREETING`) so it is **not** built here. - **D** register `src/host_cpu.cpp`, `src/free-store.cpp`, `src/exception_handler.cpp` in the port INTERFACE target. - **F/G** `exception_handler.cpp`: remove a redundant `port_cpu_id` decl; add a port-side weak definition for the test-board `g_core_stage[]` symbol. - **H** kernel `os-core.cpp` / `os-thread.cpp`: wrap the `port_cpu_id` redundant decls with `#pragma GCC diagnostic ignored "-Wredundant-decls"`. - **I/J** posix-arch clang `-Weverything` hardening: port headers add `-Wc++98-compat-pedantic` / `-Wreserved-identifier` / `-Wunsafe-buffer-usage`; the SMP `.cpp` files get a comprehensive clang suppression block before their includes.

Part C — new architectures, platforms, tests (Steps 24–31, add-only)

These steps add **new files/directories** (no existing upstream file is modified; the frozen-test-framework rule in §2.7 is enforced), so they are listed by group rather than line-by-line.

Step	Adds (new files from <code>origin/smp</code> , unless noted)
24	Port releases — no file lift; version bump + tag for the cortexm/posix-arch ports (<code>release-port.sh</code>).
25	New cores/boards in the arch repos (aarch32/aarch64 ports + new cortexm boards), brought in whole from their <code>origin/smp</code> .
26	Modular CMake: additive kernel targets in <code>CMakeLists.txt</code> — <code>micro-os-plus::iii-core</code> , <code>::port-smp-decls</code> , <code>::test-support</code> ; new <code>tests/platforms/2xcortex-m33/</code> + <code>tests/device-qemu-cortexm-m33/</code> (dual-core M33).
27	New test sources: <code>tests/sources/fp-switch/</code> ; <code>tests/smp-support/</code> (the smp branch's root <code>test_smp/</code> dissolved — <code>hw_result.hpp</code> , <code>board-contract.cpp</code> , the three runners; see the Step 27 note and <code>absorb-test-smp.sh</code>); smp tests 0–5; QEMU MPS2 AN505/AN521 linker scripts.
28	Add-only platforms: <code>tests/platforms/cortexm-pico2/</code> and the aarch32/aarch64 QEMU <code>raspi3b</code> platforms.
29	Documentation.
30	Final local merge of the step chain onto <code>xpack-development</code> ; PR is manual.
31	<code>cortexm-pico2</code> (RP2350, generic-m7 QEMU) platform — add-only.

Part 0 — Dissolve the `devices` Repository (do this first)

Problem. `origin/xpack-development` has **six** components; the `smp` branch adds a **seventh**, `micro-os-plus-iii-devices`. Upstream has no such repo and no place in its dependency graph for one. Landing SMP as "add a new repo everyone must now depend on" is exactly the kind of structural change upstream review resists — and it has nowhere to be forked *from* on `xpack-development`. So before Step 1, the `devices` repo is **folded into the architecture repos that already own its code**. Afterwards the integration proceeds with the same six repos upstream already has (plus the arch repos, which are themselves add-only in Part C).

0-A. What is actually in `devices`

Two cleanly separable kinds of content — the repo's own CMake says as much ("One copy serves every architecture: measured identical across AArch32 and AArch64"):

Kind	Paths	Nature
Arch-neutral drivers	<code>include/</code> (<code>sd.hpp</code> , <code>fatfs.hpp</code> , <code>usb_dwc2.hpp</code> , <code>fatfs_hw.hpp</code>), <code>src/</code> (<code>sd.cpp</code> , <code>fatfs.cpp</code> , <code>usb_dwc2.cpp</code>), <code>fatfs/</code> (ChaN FatFs + glue)	Portable C/C++; no ISA. Used by boards, not by the RTOS.
Per-SoC silicon	<code>soc/bcm2837/</code> , <code>soc/rk3506/</code> , <code>soc/rp2350/</code> , <code>soc/stm32f4xx/</code> , <code>soc/native/</code>	Each SoC is owned by exactly one architecture.

0-B. Destination mapping (each SoC has exactly one home)

<code>devices/...</code>	→ Destination repo(s)	→ Path	CMake target kept via ALIAS
<code>include/</code> + <code>src/</code> + <code>fatfs/</code>	aarch32 , aarch64 , posix-arch	<code>drivers/...</code>	<code>micro-os-plus::devices</code> , <code>::devices-hostfile</code>
<code>soc/bcm2837/</code>	aarch32 + aarch64 (RPi3B / Zero 2W, both widths)	<code>soc/bcm2837/</code>	<code>::soc-bcm2837</code>
<code>soc/rk3506/</code>	aarch32 (Luckfox Lyra, Cortex-A7)	<code>soc/rk3506/</code>	<code>::devices-rk3506</code>
<code>soc/native/</code>	posix-arch (host image-file SD backend)	<code>soc/native/</code>	<code>::devices-hostfile</code>
<code>soc/rp2350/</code>	cortexm (Pico 2 — headers + <code>system_rp2350.c</code>)	<code>soc/rp2350/</code>	<code>::soc-rp2350</code>
<code>soc/stm32f4xx/</code>	cortexm (nucleo/weact F411/ F412)	<code>soc/stm32f4xx/</code>	<code>::soc-stm32f411xe</code> , <code>::soc-stm32f412rx</code>

Notes: - `bcm2837` is duplicated into **aarch32** and **aarch64** on purpose. The BCM2837 mailbox support is 3 small files shared by both widths; duplicating keeps each arch repo self-contained (no cross-repo dependency), which is the whole reason we are removing the shared repo. The two copies are byte-identical. - `cortexm` receives no drivers, only its two SoCs. Per the `devices` CMake, the SD/FatFs/flatfs stack is not exercised on Cortex-M targets. - The **dual-core** RP2350 bits (SIO spinlocks, inter-core FIFO, bootrom core-1 launch) are **not** in `soc/rp2350/` — they already live in the `cortexm` board tree, because they *are* that board's SMP kernel lock and IPI (Step 25).

0-C. Migrating with history preserved

Use `git subtree split` so each subfolder carries its commit history into the destination — the script `../scripts/smp/migrate-devices.sh` does exactly the mapping above:

```
# devices, aarch32, aarch64 must be cloned alongside K/P/C (bootstrap clones the
# core three; clone the rest first – see §10.5).
scripts/smp/migrate-devices.sh
```

Mechanically, per subfolder:

```
# in the devices repo – carve out one subtree with its history:
git -C "$WORK/micro-os-plus-iii-devices" subtree split -P soc/bcm2837 -b split/soc_bcm2837
# in each destination repo – graft it under the target prefix:
git -C "$WORK/micro-os-plus-iii-aarch32" subtree add -P soc/bcm2837 \
    "$WORK/micro-os-plus-iii-devices" split/soc_bcm2837 \
    -m "migrate(devices): soc/bcm2837 -> soc/bcm2837 (history preserved)"
```

If you do **not** need history, a plain `git mv` /copy into each destination plus a one-line provenance note in each folder's README is enough; the subtree route is preferred because it keeps `git blame` / `log` intact for the drivers.

0-D. CMake shim so no consumer breaks

Each destination gets a small `CMakeLists.txt` (or a fragment `include()` d by the repo's existing one) that re-declares the **same target names** the old `devices` repo exported, now pointing at the local paths. Because the names are unchanged, every board and test that said `target_link_libraries(app PRIVATE micro-os-plus::devices)` keeps working:

```
# aarch32/drivers/CMakeLists.txt (add_subdirectory'd by the aarch32 project)
add_library(micro-os-plus-iii-devices-interface INTERFACE)
target_include_directories(micro-os-plus-iii-devices-interface INTERFACE
    "${CMAKE_CURRENT_LIST_DIR}/include" "${CMAKE_CURRENT_LIST_DIR}/fatfs")
target_sources(micro-os-plus-iii-devices-interface INTERFACE
    src/sd.cpp src/flatfs.cpp src/usb_dwc2.cpp
    fatfs/ff.c fatfs/diskio_sd.cpp fatfs/fatfs_hw.cpp)
add_library(micro-os-plus::devices ALIAS micro-os-plus-iii-devices-interface)
```

```
# aarch32/soc/bcm2837/CMakeLists.txt
add_library(micro-os-plus-iii-soc-bcm2837-interface INTERFACE)
target_include_directories(micro-os-plus-iii-soc-bcm2837-interface INTERFACE
    "${CMAKE_CURRENT_LIST_DIR}/include")
target_sources(micro-os-plus-iii-soc-bcm2837-interface INTERFACE src/mailbox.cpp)
add_library(micro-os-plus::soc-bcm2837 ALIAS micro-os-plus-iii-soc-bcm2837-interface)
```

Lift each fragment from the original monolithic `micro-os-plus-iii-devices/CMakeLists.txt` (targets `::devices`, `::devices-hostfile`, `::devices-rk3506`, `::soc-bcm2837`, `::soc-stm32f411xe`, `::soc-stm32f412rx`, `::soc-rp2350`), splitting it per destination.

0-E. Verify the migration, then archive `devices`

1. For each arch repo, build one board that uses a migrated target (e.g. aarch32 `test/rpi3b`, posix-arch `test/native` flatfs) and confirm CMake still resolves `micro-os-plus::devices` / `::soc-bcm2837` / `::devices-hostfile`.
2. Once all destinations build, the `devices` repo is **no longer a dependency**. Do not delete it — mark it archived (its history now lives, grafted, in the arch repos) and remove any `add_subdirectory(micro-os-plus-iii-devices)` / xpm dependency from board CMake and `package.json`.
3. Update §0.1's repo list mentally: the integration now tracks **6 upstream repos + arch repos**, with no standalone `devices`.

Ordering. Part 0 is a one-time restructuring done on the arch repos before the step loop. It is not one of the 30 numbered steps and has no 72-test gate of its own (the arch repos are not in that matrix); its gate is "every board that linked a `devices` target still configures and builds." Do it, verify it, then start Step 1.

0. Model of Operation

0.1 Repositories in play

```
$WORK = ~/Documents/Work/micro-os-plus
├─ micro-os-plus-iii          (K)  kernel — 30 step branches live here
├─ micro-os-plus-iii-posix-arch (P)  host port — touched in steps 13–17,19,21,23,24,25
├─ micro-os-plus-iii-cortexm  (C)  M-profile port — steps 13,15,16,18,19,21,24,25
├─ micro-os-plus-iii-aarch32  (A32) added in Part C; absorbs devices drivers + bcm2837 + rk3506
├─ micro-os-plus-iii-aarch64  (A64) added in Part C; absorbs devices drivers + bcm2837
├─ micro-os-plus-iii-devices  (DEV) DISSOLVED in Part 0 — folded into the arch repos
└─ micro-os-plus-iii-riscv    (RV)  added in Part C
```

`devices` is not carried as a 7th dependency. **Part 0** (below) folds it into the architecture repos before Step 1, so the series tracks the same six repos upstream already has plus the add-only arch ports.

0.2 The branch-per-step model

Every step is one branch in each repo it touches, named `step/NN`, forked from the previous step, or — for the first step in that repo — from that repo's `origin/xpack-development`. This keeps history **bisectable** and each PR reviewable in isolation.

```
origin/xpack-development —► step/01 —► step/02 —► ... —► step/30 —► (final merge → smp-
upstream PR)
```

The invariant at every arrow: **72/72 tests green**, and for Part B additionally **zero single-core delta** under `unifdef -UOS_USE_SMP_SCHEDULER`.

Which repos get per-step integration. Three repos have an `xpack-development` baseline that smp is integrated *into*, step by step, in sync:

Repo	Status	Per-step integration?	Base of <code>step/01</code>
<code>micro-os-plus-iii</code> (kernel)	released	yes	<code>origin/xpack-development</code>
<code>micro-os-plus-iii-cortexm</code>	released v1.1.0	yes (touched from Step 13)	<code>origin/xpack-development</code>
<code>micro-os-plus-iii-posix-arch</code>	released v1.0.1	yes (touched from Step 13)	<code>origin/xpack-development</code>
<code>micro-os-plus-iii-aarch32</code> / <code>-aarch64</code>	new (not upstream)	no — add-only in Part C	n/a

The port **release tags are not the base**. Each port's `xpack-development` HEAD is *ahead* of its last release (cortexm: branch `687e975` vs tag `v1.1.0` `8b4ae82`, smp +53; posix-arch: branch `86a6a1f` vs tag `v1.0.1`, smp +18), and that branch is the integration target. The tags are only touched at Step 24, where `release-port.sh` cuts the *next* releases (posix-arch v1.1.0, cortexm v1.2.0). `aarch32` / `aarch64` are new ports — no `step/NN` branches, no integration; they land add-only in Part C (§6).

0.3 One step, four commands

```
scripts/smp/new-step.sh      NN      # create step/NN branches in the repos it touches
# ... apply the edits for step NN (Sections 4–7 below) ...
scripts/smp/verify-step.sh  NN      # link local ports + run the 72-test gate (+ unifdef for 14–23)
scripts/smp/advance-step.sh NN      # commit, tag step-NN-green, prepare step/NN+1
```

Sections 1–3 build the automation. Sections 4–7 are the per-step edits.

1. Prerequisites & One-Time Setup

1.1 Toolchain check

Run once; it prints what the gate needs and flags anything missing.

```
# scripts/smp/check-env.sh
```

Required on `PATH`: `xpm`, `cmake`, `ninja`, GCC 11/12/13/14, Clang 16/17/18/19, `qemu-system-arm`, `unifdef`, `git`, and for the docs `python3` (+ `markdown`, `weasyprint`) or `typst`.

Known-good Node / xpm — read before the first build. The whole local-port dev loop turns on `xpm link` (`link-ports.sh` → `xpm link` → `xpm run link-deps`). Some Node/xpm combinations break it: on **Node v26.3.1 with xpm 0.23.2**, `xpm link` aborts with `TypeError: jsonPackage.isNpmPackage is not a function`, so no config can link its local ports and the gate cannot run. Use a combination where `xpm link` works — a current **LTS Node** (bleeding-edge majors like 26 are the usual culprit) with an up-to-date `xpm` (`npm i -g xpm@latest`). Verify with a one-line smoke test before starting:

```
sh node --version; xpm --version ( cd "$WORK/micro-os-plus-iii-posix-arch" && xpm link ) # must succeed, no TypeError
```

`check-env.sh` confirms the tools are *present*; this smoke test confirms `xpm link` actually *works*, which presence alone does not guarantee.

1.2 Register the working copies

```
export WORK="$HOME/Documents/Work/micro-os-plus"
cd "$WORK/micro-os-plus-iii" && git fetch origin
# Ports integrate into their OWN xpack-development branch (not the release tag):
cd "$WORK/micro-os-plus-iii-posix-arch" && git fetch origin xpack-development
cd "$WORK/micro-os-plus-iii-cortexm" && git fetch origin xpack-development
```

1.3 Create the automation directory

All helper scripts live in `scripts/smp/` inside the kernel repo (they are add-only tooling, never shipped upstream — keep them out of the final PR, see Step 30):

```
mkdir -p "$WORK/micro-os-plus-iii/scripts/smp"
```

2. The Automation Scripts

These files exist under `../scripts/smp/` (see its `README.md`) — they are the "scripts for facilitating tasks" and the "automatic updates" the plan calls for. The listings below are the reference; the checked-in files are canonical.

2.0 Script index — what each one does and how to run it

Every script sources `common.sh` and reads `$WORK` from the environment. Run them from the kernel repo root (`$WORK/micro-os-plus-iii`). None push; all are local.

Script	Invocation	What it does
<code>common.sh</code>	<i>(sourced, not run)</i>	Shared config: paths, branch refs, 12 build configs, per-step repo map.
<code>check-env.sh</code>	<code>scripts/smp/check-env.sh</code>	Verify toolchain (xpm, cmake, ninja, GCC 11–14, Clang 16–19, qemu, undef).
<code>bootstrap.sh</code>	<code>WORK=... bash .../bootstrap.sh</code>	From an empty <code>\$WORK</code> : clone the repos, fetch both branches, <code>xpm install</code> . §10.2
<code>migrate-devices.sh</code>	<code>scripts/smp/migrate-devices.sh</code>	Part 0 , once before Step 1: fold <code>devices</code> into the arch repos (history-preserving).
<code>absorb-test-smpl.sh</code>	<code>scripts/smp/absorb-test-smpl.sh</code>	Step 27 : dissolve the smp branch's root <code>test_smpl/</code> into <code>tests/smp-support/</code> and delete the folder, then repoint the aarch32/64 board run-wrappers at it. Lifts from <code>origin/smp</code> ; fail-loud + idempotent. See Step 27.
<code>integrate-aarch-harness.sh</code>	<code>scripts/smp/integrate-aarch-harness.sh</code>	Steps 25/26/28 : wire the AArch32/64 ports into the standard xPack test chain — modular kernel sub-targets (<code>iii-posix-io</code> / <code>iii-semihosting</code> / <code>iii-newlib-reent</code>), integration-model port CMake + test builder, the four harness platforms, and the <code>aarch{32,64}-rpi3b</code> build configs. Lifts from <code>origin/smp</code> ; fail-loud + idempotent. See Step 28.
<code>new-step.sh</code>	<code>scripts/smp/new-step.sh NN</code>	Create <code>step/NN</code> branch(es) forked from <code>step/NN-1</code> (or each repo's <code>origin/xpack-development</code>).
<code>show-chunk.sh</code>	<code>scripts/smp/show-chunk.sh NN <file...></code>	Print the <code>xpack-development..smp</code> diff for files, to lift the chunk.
<code>check-pristine.sh</code>	<code>scripts/smp/check-pristine.sh [NN]</code>	Enforce the frozen-test-framework rule (§2.7). Run by verify + advance.
<code>link-ports.sh</code>	<code>scripts/smp/link-ports.sh</code>	Link local port copies across all 24 configs (explicit loop).
<code>verify-step.sh</code>	<code>scripts/smp/verify-step.sh NN</code>	Gate : pristine → undef (14–23) → link → <code>test-all</code> (72) → NCPU=2 smoke (23).
<code>advance-step.sh</code>	<code>scripts/smp/advance-step.sh NN ["msg"]</code>	Pristine check, commit each touched repo, tag <code>step-NN-green</code> .
<code>run-loop.sh</code>	<code>scripts/smp/run-loop.sh [FROM] [TO]</code>	Drive new→apply→verify→advance over a range. <code>AUTO=1</code> cherry-picks <code>smp-step-NN</code> .

Script	Invocation	What it does
<code>release-port.sh</code>	<code>scripts/smp/release-port.sh</code> <code><dir> <ver> "<notes>"</code>	Part C: bump + tag a port release (edits <code>version</code> in-place; never <code>npm version</code>).
<code>full-verify.sh</code>	<code>scripts/smp/full-verify.sh</code>	MANDATORY full-matrix gate: real <code>xpm install → link-deps → prepare → build → test</code> for every platform, one gcc + one clang. See §3.1.
<code>newer-toolchains.sh</code>	<code>scripts/smp/newer-toolchains.sh</code>	Baseline compat so the tests also build on host gcc/clang newer than the pinned ones (verified gcc 16.2, clang 22.1). See §3.2.
<code>build-aarch64-rpi.sh</code>	<code>scripts/smp/build-aarch64-rpi.sh</code> (<code>BOARD=rpi3b\ rpi-zero-2w</code>)	Build+run the AArch64 (ARMv8-A) SMP port on QEMU raspi3b, 4-core; <code>smp_test0–4</code> . See §6.1.
<code>build-aarch32-rpi.sh</code>	<code>scripts/smp/build-aarch32-rpi.sh</code> (<code>BOARD=rpi3b\ rpi-zero-2w</code>)	Same for AArch32 (ARMv7-A), reached via the AArch64→32 shim. See §6.1.
<code>finalize.sh</code>	<code>scripts/smp/finalize.sh</code>	Step 30: merge baseline, drop dev tooling, run the gate (local; no push).

Typical order: `check-env` → `bootstrap` → `migrate-devices` → then per step `new-step` → `show-chunk` → *(apply chunk)* → `verify-step` → `advance-step`, or the whole range via `run-loop`; then **`full-verify` (§3.1) before you trust any "green"**; `finalize` closes Step 30. Full walk-through in §10.

2.1 `common.sh` — shared config

```
# scripts/smp/common.sh — sourced by every helper
set -euo pipefail
WORK="${WORK:-$HOME/Documents/Work/micro-os-plus}"
K="$WORK/micro-os-plus-iii"
P="$WORK/micro-os-plus-iii-posix-arch"
C="$WORK/micro-os-plus-iii-cortexm"
UPSTREAM="origin/xpack-development"

# The 12 build configs (× debug/release = 24 builds = 72 test runs).
CONFIGS=(
    native-cmake-gcc11 native-cmake-gcc12 native-cmake-gcc13 native-cmake-gcc14
    native-cmake-clang16 native-cmake-clang17 native-cmake-clang18 native-cmake-clang19
    qemu-cortex-m0-cmake-gcc qemu-cortex-m3-cmake-gcc
    qemu-cortex-m4f-cmake-gcc qemu-cortex-m7f-cmake-gcc
)

# Which repos each step touches (K always; add P/C when the step needs the port).
# Space-separated repo roots.
step_repos() {
    case "$1" in
        13|15|16|18|19|21) echo "$K $P $C" ;;
        14|17|23)          echo "$K $P" ;;
        24|25)             echo "$K $P $C" ;;
        *)                 echo "$K" ;;
    esac
}

log() { printf '\033[1;36m>>> %s\033[0m\n' "$*"; }
ok()  { printf '\033[1;32m[+] %s\033[0m\n'  "$*"; }
die() { printf '\033[1;31m[-] %s\033[0m\n'  "$*" >&2; exit 1; }
```

2.2 `new-step.sh` — branch creation

```
#!/usr/bin/env bash
# scripts/smp/new-step.sh NN — create step/NN branches in every repo it touches.
source "$(dirname "$0")/common.sh"
NN="$(printf '%02d' "${1:?usage: new-step.sh NN}")"
PREV="$(printf '%02d' "${10#$NN} - 1")"

for repo in $(step_repos "$1"); do
    cd "$repo"
    # Base: previous step branch if it exists here, else the correct upstream base.
    if git rev-parse --verify --quiet "step/$PREV" >/dev/null; then
        base="step/$PREV"
    elif [ "$repo" = "$P" ]; then base="v1.0.1"
    elif [ "$repo" = "$C" ]; then base="v1.1.0"
    else base="$UPSTREAM"
    fi
    git switch -c "step/$NN" "$base" 2>/dev/null || git switch "step/$NN"
    ok "$(basename "$repo"): step/$NN ← $base"
done
```

2.3 `link-ports.sh` — the correct 24-config link loop

Uses the explicit loop (never `link-deps-all`, which drops `native-cmake-gcc14-debug`).

```
#!/usr/bin/env bash
# scripts/smp/link-ports.sh — bind local port working copies into all 24 configs.
source "$(dirname "$0")/common.sh"
cd "$P" && xpm link
cd "$C" && xpm link
cd "$K/tests"
for c in "${CONFIGS[@]"; do
    for t in debug release; do
        xpm run link-deps --config "${c}-${t}" >/dev/null 2>&1 \
            || echo "warn: link-deps ${c}-${t} skipped"
    done
done
ok "local ports linked across 24 configurations"
```

2.4 `verify-step.sh` — the gate

Supersedes the top-level `scripts/verify-step.sh`; adds the config-scoped link and a Part-B smoke check.

```

#!/usr/bin/env bash

# scripts/smp/verify-step.sh NN – full acceptance gate for one step.
source "$(dirname "$0")/common.sh"
NN="{1:?usage: verify-step.sh NN}"

# Stage 1 – Part B single-core invariant (steps 14-23): unifdef must be identical.
if [ "$NN" -ge 14 ] && [ "$NN" -le 23 ]; then
    log "[1/3] unifdef -UOS_USE_SMP_SCHEDULER single-core invariant"
    cd "$K"; bad=0
    for f in $(git diff --name-only "$UPSTREAM" HEAD -- include/ src/); do
        [ -f "$f" ] || continue
        git show "$UPSTREAM:$f" 2>/dev/null | unifdef -UOS_USE_SMP_SCHEDULER >/tmp/o 2>/dev/null || true
        git show "HEAD:$f" 2>/dev/null | unifdef -UOS_USE_SMP_SCHEDULER >/tmp/n 2>/dev/null || true
        diff -u /tmp/o /tmp/n >/tmp/d 2>&1 || { echo "DIVERGENCE in $f"; cat /tmp/d; bad=$((bad+1)); }
    done
    [ "$bad" -eq 0 ] || die "$bad file(s) changed single-core output"
    ok "zero single-core delta"
fi

# Stage 2 – link local ports.
log "[2/3] linking local development ports"
"$K/$(dirname "$0")/link-ports.sh"

# Stage 3 – the official 72-test suite.
log "[3/3] xpm run test-all (24 builds / 72 runs, -Werror)"
cd "$K/tests" && xpm run test-all

ok "STEP $NN PASSED (72/72)"

```

2.5 `advance-step.sh` — commit, tag, hand off

```
#!/usr/bin/env bash
# scripts/smp/advance-step.sh NN — commit each repo, tag the green point.
source "$(dirname "$0")/common.sh"
NN="$(printf '%02d' "${1:?usage: advance-step.sh NN}")"
MSG="${2:-step $NN}"

for repo in $(step_repos "$1"); do
    cd "$repo"
    git add -A
    git diff --cached --quiet || git commit -m "smp-step($NN): $MSG"
    git tag -f "step-$NN-green"
    ok "$(basename "$repo"): committed + tagged step-$NN-green"
done
echo "Next: scripts/smp/new-step.sh $((10#$NN + 1))"
```

2.6 Everything is local — no push

None of these scripts push, open a PR, or write to any remote. They only `git clone` / `fetch` (read) and create branches, commits and tags **locally**. The green gate is enforced by refusing to `advance-step.sh` on red (and by you not running it); the `step-NN-green` tags are local markers. Publishing, if and when you want it, is a manual `git push` you type yourself — never automated here.

2.7 Pristine test framework — the hard rule the scripts enforce

The `xpack-development` test framework must stay still: **existing entries are never modified or deleted**. What a step may add depends on its part, and `check-pristine.sh` enforces it by comparing each touched repo's working tree to `origin/xpack-development`.

Parts A/B (steps 1–23) — total freeze. No add / modify / delete of any of: `package.json`, `package-lock.json`, anything under `tests/`, `.github/**`. Steps here only edit library sources (`include/`, `src/`, a port's `include/` / `src/`). A library `CMakeLists.txt` / `*.cmake` *outside* `tests/` may change only additively.

Part C (steps 24–28) — additions allowed, existing entries preserved. New tests, platforms and drivers arrive here, so the rule relaxes from "frozen" to "append-only", checked precisely:

Target	Add new	Modify existing	Delete/rename
<code>package.json</code> , <code>package-lock.json</code>	new keys/entries	only if every existing key & value is preserved (semantic JSON subset)	
harness wiring: <code>tests/CMakeLists.txt</code> , <code>tests/cmake/**</code>		only additively (no removed lines)	
existing test content: <code>tests/sources/<t>/</code> , <code>tests/platforms/<p>/**</code>	new files/dirs		
library <code>CMakeLists.txt</code> / <code>*.cmake</code> (outside <code>tests/</code>)		additively	
<code>.github/**</code>			(restored only by <code>finalize.sh</code> at Step 30)

So adding an xpm action for a new suite is fine (a new `"test-smp-all"` key), but touching an existing action's value fails; adding a `tests/platforms/native-smp/` is fine, but editing `tests/platforms/qemu-cortex-m0/...` fails.

One exception, added in the dry run: a package.json's top-level `version`. Step 24's `release-port.sh` bumps `posix-arch` to `v1.1.0` and `cortexm` to `v1.2.0`, which changes an existing value — the semantic-subset check flagged it as `$.version: value changed`. But a port's own version *is* the release mechanism, not a frozen test-framework entry, so `check-pristine.sh` now exempts the top-level `version` key (only that key; every other script/action/config value stays frozen). Verified both ways: the version bump passes, a change to any `scripts/` / `actions/` / `buildConfigurations` entry still fails.

Part D (steps 29–30) is not gated — Step 30 legitimately restores `.github/**` and merges the baseline.

This is deliberately stricter than the `smp` branch, which *rewrites* `tests/package.json`, `tests/CMakeLists.txt`, `tests/cmake/tests-main.cmake`, the root `CMakeLists.txt` and *deletes* `.github/workflows/ci.yml` — none of which a lifted chunk may carry over. The guard runs as **Stage 0 of `verify-step.sh`** (before the build) and inside `advance-step.sh` (before the commit), so a forbidden change can neither pass the gate nor be committed. The JSON subset check uses `python3`; verified cases: additive action , changed existing value , additive harness append , harness line removed , new platform dir , edited existing platform .

2.8 Make all scripts executable

```
chmod +x scripts/smp/*.sh
```

3. The Verification Gate (reference)

Read this first — FAST is for iterating, not for trusting. `verify-step.sh` has a `FAST=1` mode (cortex-m4f + native gcc14 only) that is fine *while developing a step*, but it builds roughly **2 of ~60 configs**. A step that is "green" under FAST is **not** verified — the other targets, every `clang` config, and every `-release` variant are simply unbuilt. In the dry run this gap was real: `qemu-cortex-m7f-release` failed to compile (the integrated kernel declared `clock_highres::has_hardware_counter()` while the build linked the *released* cortexm that does not define it) even though FAST was green. **Never report an integration as done on FAST evidence.** The mandatory acceptance gate is §3.1.

Every `verify-step.sh` run must end green on this matrix (see the plan §3):

Host builds (posix-arch)	QEMU builds (cortexm)
GCC 11/12/13/14 ×	Cortex-M0/M3 ×
Clang 16/17/18/19 ×	Cortex-M4F/M7F ×

Per build, three test executables run: `rtos-apis`, `mutex-stress`, `cmsis-os-validator` → **24 builds × 3 = 72 runs**.

Versioned configs only — not the system compiler. The 24 configs are the *versioned* toolchains `native-cmake-gcc{11,12,13,14}` and `native-cmake-clang{16,17,18,19}` (each × debug/release) plus the four QEMU Cortex-M configs. The unversioned `native-cmake-gcc-debug` / `native-cmake-clang-debug` (and `native-cmake-sys-*`) pick up **whatever cc / clang is on PATH** and are **not** part of the gate — handy for a quick local smoke build, but their result is meaningless for acceptance. A dry-run build through the unversioned `-gcc-` config on a host with GCC 16.2.1 / glibc 2.44, for instance, fails Step 1's `timegm.c` on a `-Werror=missing-prototypes` that the sanctioned GCC 11–14 do not raise (see §4 Step 1). Always gate with `xpm run test-all`, which drives the versioned matrix; install the xPack GCC 11–14 / Clang 16–19 toolchains for it.

Pass criteria per step (every step first passes the pristine check of §2.7 — the frozen test framework is untouched):

- Part A (1–13): pristine + 72/72 green with `-Werror`.
- Part B (14–23): pristine + 72/72 green **and** `unifdef` single-core invariant.
- Part C (24–28): pristine + 72/72 green (old suite unmodified) **plus** the newly added SMP tests green.
- Part D (29–30): clean doc build; full ecosystem `test-all` + `test-smp-all`.

Advance rule. Do not run `advance-step.sh NN` until `verify-step.sh NN` exits 0. Everything stays local (§2.6); nothing is published automatically.

Fast iteration (`FAST=1`). Running the full 72 for every trial is slow. For quick kernel/core iteration, `FAST=1 scripts/smp/verify-step.sh NN` runs **one real ARM target + one host config** — `qemu-cortex-m4f` (arm-none-eabi, under QEMU) plus `native-cmake-gcc14`, debug+release each (4 builds / 12 runs) — via `test-qemu-cortex-m4f-cmake` + `test-native-cmake-gcc14`. It still runs the pristine and undef checks. The **Cortex-M4F leg is the important one for kernel work**: it uses newlib (so host-libc quirks like glibc-2.44 `timegm` never arise) and catches 32-bit issues the x86 host build misses — e.g. `-Wcast-align` on allocator pointer arithmetic. The native gcc14 leg adds the host `-Werror` coverage the official gate also wants. Toolchains are xPack-pinned; the machine's own gcc/clang are **not** used (too new for µOS++ — libstdc++ 16 rejects `os::estd::chrono::ceil`, glibc 2.44 breaks strict-flag `timegm`). It is **not acceptance**: clang and the other Cortex-M targets don't run, so a step is only truly green after a plain `verify-step.sh NN` (full 72). The script warns and does not claim "72/72" in FAST mode.

3.1 The full-matrix gate — `full-verify.sh` (MANDATORY)

`verify-step.sh` proves one *step* in isolation; `full-verify.sh` proves the *whole integration builds and runs* across every platform, using the **real test-framework tools in the right order** — the same commands the VS Code "xPack C/C++ Managed Build" extension runs, so what passes here passes there too. It is the acceptance gate: the integration is **not** done until it is green.

One gcc + one clang. To keep the matrix tractable it uses exactly one host GCC (14) and one host Clang (19) — not all four of each — plus the single cross toolchain every target already has (arm-none-eabi 15 for Cortex-M/M33, the aarch cross GCCs for those ports) and the pinned QEMU. That is enough to exercise every *code path and platform* without rebuilding the same sources under eight nearly identical host compilers.

What it does, per config, in order (nothing is assumed by analogy):

```
(once) per integrated port: xpm install ; xpm link      # register the step/NN dev links
(per C) xpm install --config C                          # deps + the local:link port
      xpm run link-deps --config C                      # overlay the dev port(s) – NOT the release
d ones
      xpm run prepare  --config C                      # cmake configure
      xpm run build    --config C                      # cmake build  (-Werror)
      xpm run test     --config C                      # ctest -LE hwd (host runs; QEMU for cross)
```

It records `PASS` / `FAIL` per config (with the failing stage and first error) and **exits non-zero if any config fails**, printing the full matrix. Platforms covered: `native` (gcc + clang), `native-smp` (dual-core host SMP), `qemu-cortex-m0/m3/m4f/m7f`, `2xcortex-m33` (dual-core), each × debug/release.

Two environment realities it absorbs so the pure xpm flow completes deterministically (both are quirks of this machine's `xpm 0.23.3` / Node 26, not of the integration):

- it links the integrated ports via `xpm link` and verifies the build tree resolves the **step/NN working copies**, never the released tarballs (the released-port fallback is exactly what broke `qemu-cortex-m7f`);

- where xpm creates a toolchain's `.bin` shims but skips the package-folder link (or doesn't overlay a dep into a config's build tree), a small idempotent `repair_tree` re-links the pinned toolchain / port / 3rd-party folders. On a healthy xpm these are no-ops.

`cortexm-pico2` (RP2350) is out of this native/cortex-m gate: RP2350 has no upstream QEMU machine (hardware-only for its SMP tests), though its portable harness suites run emulated on `mps2-an500` (§6, cortexm-pico2 platform). `aarch32/64-*` are covered by their own gate (§6.1) — they are a different toolchain+machine family (arm-none-eabi / aarch64-none-elf on QEMU `raspi3b`), driven by `build-aarch{32,64}-rpi.sh`, not by this host/cortex-m `full-verify`.

Dry-run result — 16/16 PASS. `native` (gcc14 + clang19), `native-smp`, `qemu-cortex-m0/m3/m4f/m7f`, `2xcortex-m33`, each × debug/release, all green via the real xpm flow. Getting here caught the two failure classes a FAST gate hides: (1) a **released-vs-integrated port** mismatch — `qemu-cortex-m7f` linked the baseline cortexm (no step-13 `has_hardware_counter`); fixed by correct dev-linking; and (2) **clang -Weverything** failures (the destination is stricter than the smp branch) in the posix-arch SMP headers/host files and two kernel files — fixed and folded into the recipes (`part-b.sh` fixes I/J, `step03.sh`, `step05.sh`).

3.2 Newer host toolchains — `newer-toolchains.sh`

The pinned gate uses xPack **gcc 14** and **clang 19**. The tests also build and run on host toolchains *newer* than those — verified with system **gcc 16.2.1** and **clang 22.1.8** (native, single- and dual-core SMP, 100%). That needed three **baseline** / **upstream** compatibility fixes (not part of the smp lift — they touch `std/chrono` and the native platform files), captured in `newer-toolchains.sh` so they are explicit and reproducible:

1. `std/chrono` depended on libc++'s internal `std::chrono::__is_duration`, which newer libc++ renamed to `__is_duration_v` (libstdc++ never had either). Replaced with a self-contained `is_duration_` trait — portable across every gcc/clang/libc++. (This was the real code break; the `ceil` errors cascaded from it.)
2. **New clang-20+ -Weverything diagnostics** firing on untouched baseline / third-party code — `-Wthread-safety-negative`, `-Wunsafe-buffer-usage-in-libc-call`, `-Wimplicit-void-ptr-cast` — suppressed on the native platforms (harmless on older clang via the existing `-Wno-unknown-warning-option`).
3. **Unwinder choice.** A self-contained LLVM clang (xPack) bundles `libunwind`; a system clang on a GNU distro does not, and the platform's bare `-lunwind` pulled the distro's *nongnu* libunwind (no `_Unwind_Resume`) → link failure. The platform now probes `clang --print-file-name=libunwind.a` and picks `-unwindlib=libunwind` (LLVM present) or `-unwindlib=libgcc` (system clang).

gcc 16.2 needed no code changes — the Part A hardening (timegm, chrono overflow) already covered what made it "too new" for the original smp branch. clang 22 is the one that needed the three fixes above; without them even the **untouched upstream baseline** fails to compile under

clang 22, so these are upstream modernizations, kept separate from the integration and outside the §2.7 pristine rule by explicit choice.

4. Part A — Uniprocessor Fixes (Steps 1–13)

Part A edits real single-core paths and is exercised directly by the 72 tests. For each step: **target files, the edit, test focus, advance**.

Each snippet is the *target-side* result. To locate the source, use `git show smpt:<path>` in the kernel repo and lift the cohesive chunk. For files edited by more than one step (`os-c-wrapper.cpp`, `os-thread.cpp`, `os-c-decls.h`, `os-core.cpp`), do **not** copy the whole smpt file — lift only this step's hunks and strip the rest (see §10.3 "Chunk curation").

Step 1 — ISO C conformance, syscall types, list iterators

Target file	Edit
<code>include/cmsis-plus/posix/dirent.h</code> (~L54–59)	Replace empty <code>struct { ; } DIR;</code> with a member: <code>typedef struct { int reserved; ... } DIR;</code>
<code>src/libc/stdlib/timegm.c</code> (~L61–66)	Guard the prototype (see code)
<code>include/cmsis-plus/utils/lists.h</code> (iterator operator++/--)	<code>node_>next</code> → <code>node_>next ()</code> , <code>node_>prev</code> → <code>node_>prev ()</code>
<code>include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h</code>	Use <code>__READ_WRITE_RETURN_TYPE</code> where newlib defines it, else <code>ssize_t</code>
<code>src/rtos/os-thread.cpp</code> (<code>this_thread::suspend</code>)	Remove <code>inline</code> so the C wrapper resolves the symbol

```
// src/libc/stdlib/timegm.c
#if defined(__GLIBC__)
# if !(defined(__USE_MISC) || (defined(__GLIBC_USE) && __GLIBC_USE (ISOC23)))
time_t timegm (struct tm* tim_p);
# endif
#elif !defined(__APPLE__)
time_t timegm (struct tm* tim_p);
#endif
```

Test focus: every config must compile (this is the `-Wextra-semi` / `-Wmissing-prototypes` / template-instantiation gate). **Advance:** `advance-step.sh 1 "ISO C, timegm, list iterators, exported suspend"`.

Verified in a dry run, plus a forward-compat caveat. Building this exact chunk (the three small files from `smp` wholesale + the one-line `inline` removal in `os-thread.cpp`) compiles clean under the sanctioned GCC 11–14. On a *newer* toolchain than the matrix — GCC 16.2.1 with glibc 2.44 — the guard above does **not** emit the prototype (glibc there takes the `__GLIBC_USE(ISOC23)` branch) while `<time.h>` still hides `timegm` under the strict `_POSIX_C_SOURCE` / `_XOPEN_SOURCE` flags, so the file fails `-Werror=missing-prototypes`. This is out of scope for the 72-run gate (GCC 11–14), but if you later target glibc $\geq \sim 2.4x$, widen the guard to also declare the prototype when `__GLIBC_USE(ISOC23)` is set but the header has not exposed it.

Step 2 — Memory: overflow, usable size, calloc/realloc

Target file	Edit
<code>include/cmsis-plus/rtos/os-memory.h</code> (<code>align_size</code> , ~L85)	Saturate: return <code>SIZE_MAX</code> when <code>size > SIZE_MAX - align + 1</code>
<code>src/memory/first-fit-top.cpp</code>	Add <code>do_usable_size()</code> ; wrap casts in <code>-Wcast-align</code> / <code>-Wunsafe-buffer-usage</code> pragmas
<code>src/memory/lfifo.cpp</code>	Reset chunk ptr to <code>nullptr</code> when first free chunk too small
<code>src/memory/block-pool.cpp</code> (L173)	Fix inverted assert: guard <code>if (res == nullptr)</code>
<code>src/libc/stdlib/malloc.cpp</code>	<code>calloc</code> overflow check; <code>realloc</code> copies <code>min(usable, new)</code>

```
// src/libc/stdlib/malloc.cpp – calloc overflow guard
if (nelem != 0 && elbytes > (SIZE_MAX / nelem)) { errno = ENOMEM; return nullptr; }
```

Test focus: `mutex-stress` + allocator paths; `-Wcast-align` on M0/M3.

Step 3 — C++17 aligned new/delete, static error category, chrono

Target file	Edit
<code>src/libcpp/new.cpp</code>	10 aligned <code>operator new/delete</code> (<code>std::align_val_t</code>) → <code>memory::alloc/free</code> , gated <code>#if __cplusplus >= 201703L</code>
<code>src/libcpp/system-error.cpp</code>	Function-local <code>static const</code> category; wrap in <code>-Wexit-time-destructors</code> pragma
<code>src/libcpp/chrono.cpp</code>	Split cycle→ns as <code>(cyc/f)*1e9 + (cyc%f)*1e9/f</code> to avoid 64-bit overflow

Step 4 — C wrapper: one-shot timer, polymorphic delete, 64-bit timeouts

`src/rtos/os-c-wrapper.cpp`: default `timer::once_initializer`; delete via `static_cast<mutex_recursive*>` when `type==recursive`; cast to 64-bit `before` `* 1000u` in the 10 CMSIS-v1 wrappers.

Step 5 — POSIX I/O thread-safety

`src/posix-io/file-descriptors-manager.cpp`: wrap `allocate/deallocate/index` in `interrupts::critical_section`; null-check slots. `file-system.h` / `net-stack.h`: lock free-list `link()` / `unlink_head()`. `block-device.cpp`: fix inverted `if (size == 0) return EINVAL;`.

Step 6 — ARMv8-M & semihosting

`include/cmsis-plus/arm/semihosting.h`, `src/startup/exception-handlers.c`: add `__ARM_ARCH_8M_MAIN__` / `__ARM_ARCH_8M_BASE__` guards + `SecureFault_Handler`. `src/semihosting/c-syscalls-semihosting.cpp`: apply `S_IFCHR` only when no type set. Add weak `os_board_console_mirror()` called from `__posix_write()`.

Step 7 — Timer ISR safety

`src/rtos/internal/os-lists.cpp`: run callback **outside** the critical section (unlink → unlock → call → relock). `src/rtos/os-timer.cpp`: re-arm at `now + period` when `timestamp + period <= now`. `os-thread.h`: static scratch `errno` in handler mode.

Step 8 — Mutex PI / ceiling

`src/rtos/os-mutex.cpp`: check ceiling *before* taking ownership; `boosted = max(boosted, waiter->priority())`; recompute owner prio on unlock and clear `boosted_prio_`; cache `owner_` across the uncritical window.

Step 9 — Thread lifecycle, `state::destroying` (7)

`include/cmsis-plus/rtos/os-c-decls.h`: add `destroying = 7`. `os-thread.cpp` / `os-idle.cpp`: single-lock join/register/suspend; implement `detach()`; restrict `resume()` to `suspended / initializing`.

Step 10 — CondVar atomicity

`src/rtos/os-condvar.cpp`: link-then-unlock-then-suspend in `wait()`; bind `timed_wait()` timeout to the attribute clock. `diag/instrumentation.h`: add `OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR (12)`.

Step 11 — `std::thread` functor lifetime

`include/cmsis-plus/libcpp/thread-cpp.h`: `native_handle()->join()` before delete. `include/cmsis-plus/std/thread_internal.h`: keep private `function_object_` for deterministic free.

Step 12 — MQueue reschedule

`src/rtos/os-mqueue.cpp`: `port::scheduler::reschedule()` after the critical section in all 6 send/receive paths.

Step 13 — Highres clock port sync (first cross-repo step: K + C + P)

- **K** `src/rtos/os-clocks.cpp`: call `port::clock_highres::has_hardware_counter()`.
- **C** `include/cmsis-plus/rtos/port/os-inlines.h`: `has_hardware_counter()→false`, `hardware_counter()→0`, and fix `cycles_since_tick()` to test `SCB->ICSR & SCB_ICSR_PENDSTSET_Msk` (not `SysTick->CTRL`).
- **P** `os-inlines.h`: `has_hardware_counter()→true`, read `CLOCK_MONOTONIC`.

```
// C: include/cmsis-plus/rtos/port/os-inlines.h
inline clock_highres::timestamp_t clock_highres::cycles_since_tick (void) {
    uint32_t load = SysTick->LOAD, val = SysTick->VAL;
    if ((SCB->ICSR & SCB_ICSR_PENDSTSET_Msk) && (val > (load / 2)))
        return (load - val) + load;
    return load - val;
}
```

Because P and C are touched, `new-step.sh 13` opens `step/13` in all three repos; `verify-step.sh` links the local ports so the kernel builds against them.

5. Part B — SMP Infrastructure (Steps 14–23)

Every change is inside `#if defined(OS_USE_SMP_SCHEDULER)`. The macro is **undefined** in the 72-test gate, so the gate proves the single-core binary is unchanged; Stage 1 of `verify-step.sh` proves it structurally with `unifdef`.

Golden rule for Part B edits: if `unifdef -UOS_USE_SMP_SCHEDULER` on your changed file differs from the same command on the upstream file, the edit leaked outside the guard. Fix it before running the build. (`sc-project.py` stands in for `unifdef` — Stage 1 of `verify-step.sh` compares each step's single-core projection against the **previous** step's, not the baseline.)

5.0 Part B is one interwoven blob — strategy

Unlike Part A (distinct edits to distinct files, naturally bisectable), all ten Part B steps' SMP code is **interleaved inside the same** `#if defined(OS_USE_SMP_SCHEDULER)` **blocks in the same files** (`os-core.cpp` carries steps 14/15/16/18/19/22 at once). The whole smp `os-core.cpp` is +135 lines but its single-core projection $\approx$ baseline (blank-line diffs only) — the SMP code is cleanly gated, but it is not ten

separable hunks, and the single-core invariant cannot verify *how* you slice the SMP code (only that single-core is unchanged).

Two workable approaches (a dry run validated the first for kernel + cortexm):

1. **Collapse (recommended).** Apply the whole-smp Part-B files as one (or a few thematic) commits — see `../scripts/smp/chunks/part-b.sh`. The single-core projection stays identical to step/13 (invariant passes) and the single-core FAST build stays green. The kernel and cortexm collapse **cleanly**. Two lift-time fixes recur (both from smp code meeting the stricter baseline gate): re-add the now-valid `os_thread_state_destroying` static_assert that Step 4 stripped, and `struct timespec` → `timespec` in posix-arch `os-inlines.h` (`-Werror=redundant-tags`).
2. **Per-step (14→23).** Faithful but labour-intensive; the invariant gives no extra safety over the collapse, since it only checks single-core.

posix-arch is genuine port engineering, not a lift. The posix-arch `src/rtos/os-core.cpp` and its port headers are **coupled** (the headers change `lock_state` to `lock_state[OS_NCPU]`, `clock::signal_number` to a function, etc.) — you cannot apply one without the other. The whole-smp posix-arch `os-core.cpp` then meets several baseline-gate / third-party incompatibilities that need real fixes, not chunk-lifting: - `context::create()` uses `ctx->uc_sigmask`, **absent from the xPack libucontext** type (glibc's `ucontext_t` has it); - `-Werror=redundant-decls` on `port_cpu_id()` / `os_systick_handler()` now that the headers declare them; - `-Werror=address` on `&lock_state[OS_NCPU]`, `signal_number` used as value vs function, `volatile bool[1]` assignment.

Budget dedicated posix-arch work for Part B; the kernel and cortexm SMP layers land by collapse, but the host port's SMP core is its own task.

How deep (measured, and now GREEN in the dry run). The kernel + cortexm collapse is fully green (single-core invariant holds, cortex-m4f + the kernel build and 60/60 tests pass). The posix-arch SMP core is **also green** on the native host (gcc14 debug **and** release: build + link clean under `-Werror`, **100% tests passed, 0 failed**). Everything below is captured in `chunks/part-b.sh` as fixes **A–G** and reproduces byte-for-byte from `origin/smp`:

1. **os-core.cpp code fixes (A–C):** `struct timespec` → `timespec` (`-Werror=redundant-tags`); drop redundant `port_cpu_id` / `os_systick_handler` forward-decls (`-Werror=redundant-decls`); guard `ctx->uc_sigmask` for the system `<ucontext.h>` type only (`#if !defined(OS_INCLUDE_LIBUCONTEXT)`) and block `irq_set` in the trampoline so a libucontext context also starts masked.
2. **new source files** it links against — `host_cpu.cpp`, `free-store.cpp` (`os_startup_initialize_hardware*`), `exception_handler.cpp` + their headers. `board-contract.cpp` is **not** built for the native host (it `#error`s without a board `PORT_GREETING`). These files are compiled **unconditionally**: the smp posix `os-core.cpp` is a *refactor* whose always-compiled `initialize()` / `switch_stacks()` / `create()` delegate to `host_cpu::...`, so they are not `#if`-SMP-gated and must always link (fix D).

3. **host_cpu.cpp fixes** (E–G): drop its own redundant `port_cpu_id` / `os_systick_handler` decls and add the missing prior declaration for the strong `port_smp_ipi` override (`-Werror=missing-declarations`) (E); the same redundant-decl strip in `exception_handler.cpp` (F); and a **port-side weak definition of** `g_core_stage[OS_NCPU]` (G) — that symbol is otherwise defined only by the SMP *test board* (`test/boards/native/src/smp.cpp`), which the pristine single-core harness does not link, so `host_cpu.cpp`'s always-compiled `install_handlers()` reference would be undefined. The weak default links in a non-SMP-test build; the board's strong definition overrides it.
4. an **additive** edit to the posix-arch `CMakeLists.txt` `target_sources(... INTERFACE)` list, registering the three new files (fix D). We deliberately do **not** apply smp's own `CMakeLists.txt` (its standalone `UOS_SMP_DIR` / sibling-`add_subdirectory` model is incompatible with the xpack-development harness that consumes the port as an INTERFACE lib) — that larger restructure is the non-additive part of **Step 26**, deferred.

Toolchain note (dry run). The native gate must use the pinned xPack **gcc 14.2.0** (`xpm install --config native-cmake-gcc14-*`), never the machine `/usr/bin/cc` — the host gcc16/binutils emit `.base64` asm directives its own `as` rejects. `libucontext` (`@xpack-3rd-party/libucontext 1.2.0`) is a normal installed dep; its `xpm link` line in the baseline `link-deps` is expected to warn ("no development link") and is tolerated. On this box xpm 0.23.3 / node 26 is flaky about materialising the config-scoped gcc xpack folder — if the build falls back to system gcc, put the xPack gcc `bin` on `PATH` for `prepare` + `build`.

So posix-arch SMP is **Part-B-and-C entangled** but does **land green now** via the additive path: fixes A–G + the additive `target_sources`. Only the full `CMakeLists.txt` restructure (modular targets) is left for Step 26. The practical order is still: kernel + cortexm Part B collapse first; posix-arch SMP rides in with the collapse and its Part C release.

Step 14 — `port_cpu_id()` (K + P)

- **K** `src/rtos/os-core.cpp`, `os-thread.cpp`: `extern "C" unsigned port_cpu_id(void);`
- **C** `os-inlines.h`: `return 0 + static_assert(OS_NCPU == 1)`.
- **P** `src/rtos/os-core.cpp`: TLS `_this_cpu`, **noinline** + memory barrier:

```
#if defined(OS_USE_SMP_SCHEDULER)
thread_local unsigned _this_cpu = 0;
extern "C" __attribute__((noinline)) unsigned port_cpu_id (void) {
    asm volatile ("" ::: "memory");    // defeat Clang 16-18 %fs:0 caching across migration
    return _this_cpu;
}
#endif
```

Step 15 — per-CPU scheduler lock `lock_state[OS_NCPU]` (K + P + C)

`os-core.cpp`: scalar → array; in P mask `SIGALRM` before reading `port_cpu_id()`.

Step 16 — recursive kernel lock `_smp_klock` (K + P + C)

`include/cmsis-plus/rtos/port/os-decls.h`: `struct smp_klock_t {atomic lock; owner; count;}`;
`_smp_klock_enter/exit` with acquire/release; owner cleared **before** the release store (see analysis §2.3). `os-c-decls.h`: `SMP_NO_OWNER 0xFFFFFFFFU`.

Step 17 — per-CPU interrupt tracking (K + P)

P `os-decls.h`: `irq_set` + `_in_isr[OS_NCPU]`; `in_handler_mode()` reads `_in_isr[port_cpu_id()]`.

Step 18 — per-CPU current thread `current_thread_[OS_NCPU]` (K + P + C)

`os-sched.h`: array; `this_thread::thread()` reads `[port_cpu_id()]` with local IRQs masked.

Step 19 — per-CPU idle threads `os_idle_thread_core[OS_NCPU]` (K + P + C)

Each core registers its private idle context at boot.

Step 20 — CPU affinity mask (K)

`os-thread.cpp`: `th_cpu_affinity` (default `0xFFFFFFFF`), `cpu_affinity()` API, main thread pinned to `1U<<0`.

Step 21 — 5-stage deferred publish/claim (K + P + C)

Claim (`to->stack_ptr=nullptr`) → spill → SP switch → publish (`from->stack_ptr=sp`, release store) → restore. Picker gate checks `state_ != running && stack_ptr != nullptr`.

Step 22 — SMP ready-list picker (K)

`os-core.cpp` `internal_switch_threads`: highest-priority thread with `(affinity & (1<<cpu))` and published stack; else `os_idle_thread_core[cpu]`.

Step 23 — POSIX host multiprocessing / IPI (K + P) — smoke test

Add P `src/host_cpu.cpp`, `include/host_cpu.hpp`: spawn `OS_NCPU` pthreads, per-CPU `timer_create(SIGEV_THREAD_ID)`, IPI via `pthread_kill(.., SIGRTMIN+1)`. K weak `port_smp_ipi(cpu)` overridden strongly in P.

Extra gate for 23: besides 72/72, build one config with `-DOS_USE_SMP_SCHEDULER -DOS_INTEGER_RTOS_PORT_NCPU=2` and run a smoke start/stop to prove the host CPUs come up. Add to `verify-step.sh` a branch:

```

if [ "$NN" -eq 23 ]; then
    log "SMP smoke: NCPU=2 host bring-up"
    cd "$K/tests" && xpm run build --config native-cmake-gcc14-debug \
        -- -DOS_USE_SMP_SCHEDULER=1 -DOS_INTEGER_RTOS_PORT_NCPU=2
fi

```

6. Part C — Releases, New Targets, Add-Only Tests (Steps 24–28)

Strictly **add-only** upstream: no existing target/runner/config is edited.

6.0 Architecture test-coverage strategy (aarch32 / aarch64 / cortex-m33)

The kernel/core is compiled by **every** target's toolchain: native (gcc/clang) for the posix build, **arm-none-eabi** for cortex-m, and the aarch32/aarch64 cross compilers for those ports. But `xpack-development` only ships **native + cortex-m** tests, so:

- **Beginning (Parts A/B):** gate on the **actual existing harness only** — native
- cortex-m. Do **not** add or fabricate aarch/m33 builds; the cortex-m (arm-none-eabi) leg already exercises the same newlib + ARM EABI and, crucially, catches 32-bit issues the x86 host build cannot — e.g. `-wcast-align` on the allocator's pointer arithmetic (a real Step 2 find that native gcc/clang missed).
- **Later (Part C):** bring the new-architecture coverage in as the **smp branch's own tests 0–5**, added add-only for aarch32 / aarch64 / cortex-m33 (Steps 27–28). These are the existing, proven smp tests — not new inventions — promoted here as new platforms and a new `test-smp-all` action, never touching the frozen suites.

So arch coverage is **deferred to Part C but reuses smp's real tests 0–5**, rather than being smoke-faked during Parts A/B or left entirely to the final merge.

6.1 New-architecture bring-up — done & verified (dry run)

The aarch32/aarch64 ports existed only on `smp` as ~100-file **standalone-model** ports (their `xpack-development` branches are empty stubs), so adding them is a *from-scratch* re-host into the xpack topology, not a template copy. Both were brought up and **run green on QEMU raspib** (4× Cortex-A53, `-smp 4`) — `build-aarch{32,64}-rpi.sh`:

Board	ISA	QEMU boot	smp_test0–4
aarch64-rpi3b	ARMv8-A	direct <code>-kernel</code>	5/5 PASS
aarch64-rpi-zero-2w	ARMv8-A	direct <code>-kernel</code>	5/5 PASS
aarch32-rpi3b	ARMv7-A	AArch64→32 <code>shim8.img</code> + <code>-device loader, addr=0x10000</code>	5/5 PASS
aarch32-rpi-zero-2w	ARMv7-A	shim	5/5 PASS

Each boots all four cores, releases the secondaries via the **spin-table**, and runs the cross-core SMP tests (semaphore ping-pong et al.) to `RESULT: PASS`.

What the re-host required (the reusable recipe):

- **Thin kernel, not fat** `::iii`. These ports own startup/trace/syscalls, so the fat kernel double-defines `trace::write`/startup. The kernel grew three **additive** targets: `micro-os-plus::iii-core` (the 37-file portable core, with the `weak` `trace.cpp` the port overrides), `::port-smp-decls` (`port/smp-common`), `::test-support` (`tests/smp-support/include`).
- **BCM2837 SoC dissolved out of** `devices` into each arch repo (realizing Part 0 — no standalone `devices` dependency).
- **free-store** / `_sbrk`: aarch64 is **not** `__ARM_EABI__`, so the kernel's EABI-only `initialise-free-store.cpp` + `_sbrk.c` don't apply — the aarch64 port supplies its own `free-store.cpp` + a `_sbrk` stub (the posix-arch pattern). aarch32 **is** EABI, so it reuses the kernel's own — no port copies.
- **Build knobs**: `-DTRACE -DSEMIHOST -DSEMIHOST_TRAP_CHOSEN -DSEMIHOST_TRAP_HLT -DQEMU_BUILD -DOS_NCPU=4 -DOS_USE_SMP_SCHEDULER=1 -DSOC_BCM2837 -DOS_SMP_IPI_SGI=0` and the board id; `QEMU_BUILD` is what routes the verdict out through semihosting and takes the clean exit path.
- **aarch32 shim**: raspi3b starts its A53s in AArch64, so the ARMv7-A image is reached through a tiny `shim8.img` (built from `qemu-raspi3-shim/shim.S`).

Still polish, not yet done: fold these into harness CMake platforms+configs and add the per-board harness suites (rtos-apis/mutex-stress/cmsis-os-validator). The hard part — four boards, two ISAs, real 4-core SMP boot — is green.

Step 24 — Port releases

Tag `posix-arch v1.1.0`, `cortexm v1.2.0` (no `devices` release — dissolved in Part 0; its drivers/SoCs now version with their host arch repo); merge upstream into `aarch32` / `aarch64`.

Do NOT bump with `npm version` (it pushes). The ports carry the upstream xPack lifecycle script `postversion: git push origin --all && git push origin --tags`, and npm runs it **even with** `--no-git-tag-version` — a hard breach of the local-only rule (verified in the dry run: it invoked the push, which only failed here for lack of remote auth). `release-port.sh` therefore edits

`package.json`'s `version` field **directly in Python** (no npm at all) and does the commit + `git tag` by hand, so nothing can reach a remote:

```
```bash
```

## scripts/smp/release-port.sh "" (local only)

```
bash scripts/smp/release-port.sh "$P" 1.1.0 "SMP host CPU model + highres clock" bash scripts/
smp/release-port.sh "$C" 1.2.0 "SMP port lock + IPI + highres ICSR" ``
```

Dry-run result: both tags created **locally only** ( `git ls-remote --tags` shows 0 on origin), working trees clean, each release commit touches only the one `version:` line. The releases land on the current step branch, which after the Part B collapse is `step/24 <- step/14` (see the `new-step.sh` note below).

### Step 25 — New cores/boards

C: `include-m33/`, `include-rp2350/`, `os-core-m33.cpp`, `os-core-rp2350.cpp` (RP2350 SIO spinlock `0xD0000100`, FIFO IRQ 25). P: `board-contract.cpp`, `free-store.cpp`, `exception_handler.{hpp,cpp}`. K: wake-up IPI in `thread::resume()` under `OS_INTEGER_RTOS_PORT_NCPU > 1`.

**Dry-run status.** The M33/RP2350 core files land **add-only** on `step/25` ( `git checkout origin/smp -- include-m33/... src/rtos/os-core-m33.cpp ...` ), and the pristine guard passes. But their *QEMU* build cannot be verified without the full Step 26 modular CMake (see the boundary note there), so it is deferred.

⚠ **Correction — the first native "NCPU=2" attempt was NOT real SMP.** Passing `-D OS_USE_SMP_SCHEDULER=1 -D OS_INTEGER_RTOS_PORT_NCPU=2` on the `cmake / xpm` command line sets them as **CMake cache variables**, which do **not** become compile definitions — the harness only turns *platform* `target_compile_definitions` into `-D` flags. So that build compiled the kernel **single-core** (no `OS_USE_SMP_SCHEDULER` reached the compiler); its green result was just the ordinary single-core suite. `verify-step.sh` Stage 4's `xpm run build ... -- -D...` has the same trap. The real native SMP build is the **native-smp platform** (Step 28), which sets the only way that reaches the compiler — via the platform's `target_compile_definitions`. See Step 28 for the genuine result (and the kernel `port_cpu_id` defect it uncovered, which the fake build hid).

### Step 26 — Modular CMake

Add `cmake/toolchains/`, `cmake/uos-app.cmake`, `port/smp-common/`; new targets `micro-os-plus::iii-posix-io`, `::iii-drivers`, alias `::iii`.

**Done & verified in the integrated topology (dry run).** The smp branch's own modular CMake is the *standalone multi-repo* model ( `UOS_SMP_DIR` / `add_subdirectory` siblings, a mandatory `micro-os-plus-iii-devices` sibling, a **thin** `::iii` plus granular `::iii-startup` / `::iii-posix-io` /... split). None of that fits xpack-development, and adopting it wholesale would break the frozen 72-suite (which links the **fat** `::iii` ) and re-introduce the `devices` repo that Part 0 dissolves. So Step 26 was done the **integrated, add-only** way:

- **cortexm port (additive):** append one target, `micro-os-plus::cortexm-qemu-m33` , to the baseline cortexm `CMakeLists.txt` (include-m33/, os-core-m33.cpp, -mcpu=cortex-m33). It links the **fat** `micro-os-plus::iii` and pulls in **no** `devices` sibling. The baseline `::iii-cortexm` above it is untouched.
- **new platform (add-only):** `tests/platforms/2xcortex-m33/` mirrors the baseline qemu-cortex-m4f platform (same `ENABLE_*` / `add_test_executable` / `common-options` +fat-`::iii` + `::platform` shape), differing only in the `mps2-an521 --cpu cortex-m33 --smp 2` SSE-200 launch. Because the platform is new, it links the **fat** `::iii` and needs **none** of the granular `::iii-*` targets — avoiding the thin/granular split entirely.
- **self-contained device (add-only):** `tests/device-qemu-cortexm-m33/` carries its own `cmsis_device.h` (the M33/`core_cm33.h` branch) + the AN521 memory map, and **reuses the generic device's startup/vectors by relative path** — except `exception-handlers.cpp` , dropped because its only symbol `SysTick_Handler` is always provided by the SMP m33 port. The shared `device-qemu-cortexm` stays **byte-identical**, so the existing m0/m3/m4/m7 platforms are provably unaffected (this is why we did NOT take the smp branch's `#if ! OS_USE_SMP_SCHEDULER` edit to the shared header).
- **config (additive):** `2xcortex-m33-cmake-gcc-debug/release` + a `test-2xcortex-m33-cmake` action + the one missing `arm-cmsis-core-dependencies` base, all appended to `tests/package.json` ; the §2.7 guard passes.

**Dry-run result — real dual-core Cortex-M33 SMP on QEMU mps2-an521 (SSE-200), arm-none-eabi 15.2.1, xPack QEMU 8.2.6, `--smp 2` :**

Test (M33 dual-core)	Result
<code>rtos-apis-test</code>	Passed
<code>mutex-stress-test</code>	Passed (cross-core contention)
<code>cmsis-os-validator-test</code>	60/60 Passed

Two M33 cores with the real SSE-200 hardware IPI (MHU) and per-core SysTick, the SMP scheduler live — committed as `step-26-green` (kernel + cortexm). The same pattern (additive port target + fat-`::iii` platform + self-contained device) is the template for `cortexm-pico2` (RP2350) and the aarch32/64 QEMU targets in Steps 27–28.

## Step 27 — New test sources (smp tests 0–5 + SMP stress)

Bring in the **smp branch's own tests 0–5** for the new architectures, plus `tests/sources/fp-switch/` (FPU switch) and `tests/smp-support/` (shared SMP test scaffolding); QEMU MPS2 AN505/AN521 dual-core linker scripts. These are existing, proven smp tests added as **new files** — no existing suite is modified (§6.0).

`test_smpl/` is dissolved, not lifted. The smp branch keeps its shared SMP test scaffolding in a root `test_smpl/` directory — `include/hw_result.hpp` (the PASS/FAIL handshake), `src/board-contract.cpp` (the compile-time board-fact contract), and the three runners `run-qemu.sh` / `run-host.sh` / `run-hw.sh` (+ `no-power-cycle/` OpenOCD configs). `xpack-development` has no such root folder, so lifting it verbatim would plant a non-upstream top-level directory. `absorb-test-smpl.sh` instead relocates every file under `tests/smp-support/` (next to `tests/sources` and `tests/platforms`), rewrites the self-referential usage examples inside the runners/configs, **deletes the root `test_smpl/`**, and repoints the aarch32/64 board run-wrappers (`test/boards/rpi-zero-2w/{qemu,hw}.sh`) — which on the smp branch exec a standalone sibling `micro-os-plus-iii-smp/test_smpl/run-*.sh` — at the kernel's `tests/smp-support/` (`UOS_SMP_SUPPORT_DIR`). The script lifts from `origin/smp`, asserts every anchor it rewrites, and fails if any `test_smpl` reference survives, so it is reproducible and idempotent.

**Done & verified (dry run) — `fp-switch` on the dual-core M33.** The FPU-context-switch stress suite lands entirely add-only: - `tests/sources/fp-switch/` copied whole from `origin/smp` (new dir: `CMakeLists.txt` exposing `test::fp-switch`, `src/main.cpp`, its `os-app-config.h`). - one **append** to the harness `tests/cmake/global-definitions.cmake` (`set(ENABLE_FP_SWITCH_TEST true)` — 0 removed lines, so the append-only harness rule passes). Existing platforms never reference `fp-switch`, so the flag is inert for them. - wired into the **new `2xcortex-m33`** platform only (its own `dependencies-folders.cmake` + an `ENABLE_FP_SWITCH_TEST` block in its `CMakeLists.txt`) — both files are Step-26 additions, so this is editing our own new files, not the frozen ones.

Dry-run result on QEMU mps2-an521 `--smp 2`: `fp-switch-test` ("FPU context switch test, 6 threads, 3000 ticks") **Passed**, alongside the other three — **4/4 green**, committed as `step-27-green`. This exercises per-core FPU state preserved across SMP context switches on real dual-FPU (SSE-200) hardware.

**Harness suites on the new AArch ports — done, through the standard chain.** The portable harness suites (`tests/sources/*`) have **no `main()`**: the smp branch supplies a **strong `main()` + startup hooks via the harness `platform-support.cpp`** (the kernel's weak `main` is deliberately not used — it never sets the interrupts stack, which leaves a 0-byte IRQ stack and hangs the scheduler), and each board's `tests.cmake` links the suite with `-WL,--wrap=os_main` around its `harness-suite.cpp` (the verdict). This mechanism already passes on the smp branch, so it is **injected from `origin/smp` and adapted to the `xpack-development` consumption model** (the smp branch uses the standalone sibling-repo harness; `xpack-development` consumes dev-linked xPacks). The adaptation — reproducible via `../scripts/smp/integrate-aarch-harness.sh` — is: three additive kernel sub-targets (`iii-posix-io`, `iii-semihosting`, `iii-newlib-reent`; `test-support` also carries `board-contract.cpp`), the `uos_add_app` helper linking the thin `iii-core`,

each port's integration-model root `CMakeLists.txt` + test builder (which skips a device test when no `devices` target exists), the four harness platforms, and the `aarch{32,64}-rpi3b` build configs in `tests/package.json`.

**Done & verified (dry run) — via the standard `xpm` chain** (`xpm run test-<suite>-qemu --config aarch{32,64}-rpi3b-cmake-gcc-debug` → `configure` → `build` → `ctest` → `run-qemu.sh`, QEMU raspib3 4-core SMP): on **both** `aarch64-rpi3b` and `aarch32-rpi3b`, `smp_test0..4`, `mutex-stress` and `cmsis-os-validator` all report `RESULT: PASS`. (The earlier apparent "mutex-stress hang" was never the suite — it was the missing `platform-support.cpp` / 0-byte IRQ stack.) `rtos-apis` and the SD/USB tests still need `run-qemu.sh`'s SD-image seeding (privileges/tools) and the device-driver dissolution — separate items, out of this scope. `build-aarch{64,32}-rpi.sh` remain a lightweight reproduction of the standalone `smp_test0..4` bring-up only.

## Step 28 — Add-only platforms

`tests/platforms/`: `2xcortex-m33`, `cortexm-pico2`, `aarch32-rpi3b`, `aarch64-rpi3b`, `native-smp`. Extend `tests/cmake/tests-main.cmake` **additively** and add a new `test-smp-all` action to `tests/package.json` (append-only — the pristine guard of §2.7 enforces that existing entries are untouched).

**Done & verified — `native-smp` (dry run), and a real kernel defect it caught.** The new add-only `native-smp` platform (mirrors the baseline `native` platform, links `micro-os-plus::iii-posix-arch` + `libucontext`, host executables run directly) sets `OS_INTEGER_RTOS_PORT_NCPU=2` + `OS_USE_SMP_SCHEDULER=1` as platform `target_compile_definitions` — the only place the harness turns into `-D` flags, so this is the **first genuinely-SMP native compile** (see the Step 25 correction). It immediately exposed a latent Part B defect: the kernel's SMP forward-decls of `port_cpu_id` (`src/rtos/os-core.cpp`, `os-thread.cpp`) are **redundant** with the `posix-arch` port's own file-scope decl (`os-inlines.h`, pulled in via `os.h`) under `-Werror=redundant-decls`. `cortexm/m33` never file-scope-declare it, so the m33 build (Steps 26–27) was clean and did not catch this. **Fix H** (now in `chunks/part-b.sh`) wraps the two kernel decls in `#pragma GCC diagnostic ignored "-Wredundant-decls"` — benign, since the redeclaration is identical — and applies only inside the existing `#if OS_USE_SMP_SCHEDULER` block, so single-core and `cortexm/m33` are unchanged (m33 re-verified 4/4 after the fix).

Dry-run result — **native host, true dual-core SMP (gcc 14.2, `native-smp` platform):** `rtos-apis-test`, `mutex-stress-test`, `cmsis-os-validator-test` (60/60) — **3/3, 100%**. Plus the additive `test-smp-all` action (`test-2xcortex-m33-cmake` + `test-native-smp`) and `native-smp-cmake-gcc14` debug/release configs; the §2.7 guard passes. Committed `step-28-green`.

**Remaining new-arch targets —** `cortexm-pico2` (RP2350) has no upstream QEMU machine (real-hardware `hwd`, so build-only), and `aarch32/64-rpi3b` need their new port repos (add-only, not yet created; the `aarch64-none-elf` + `qemu-arm raspib3` toolchains are present). Both follow the Step 26 template verbatim (additive port target + fat-`::iii` platform + self-contained device).

Gate for Part C: the original `xpm run test-all` (72) still green and `xpm run test-smp-all` green.

---

## 7. Part D — Docs & Final Merge (Steps 29–30)

---

### Step 29 — Documentation

Render this runbook and the companions:

```
cd docs && ./render-pdfs.sh Implementation-SMP-Integration \
 --title "SMP Integration – Implementation Runbook" \
 --subtitle "Concrete, verifiable execution of the 30-step plan" \
 --footer "micro-os-plus-iii-smp"
```

(Add the matching `render Implementation-SMP-Integration ...` line to `render-pdfs.sh` so it renders reproducibly.)

### Step 30 — Final merge (local); PR is manual

1. Merge latest `origin/xpack-development` into the accumulated step branch.
2. Restore upstream metadata: `.github/workflows/ci.yml`, `README.md`, `LICENSE`, Doxygen/ template dirs.
3. Remove local symlinks in `tests/xpacks/`; remove `scripts/smp/` (dev-only).
4. Final gate: `xpm run test-all` (72) and `xpm run test-smp-all`.
5. Everything is now green on the **local** `step/30` branch. Pushing and opening the PR are manual actions you take by hand — no script does them.

Final-merge helper — `../scripts/smp/finalize.sh`, local only: merges baseline into `step/30` in **every** touched repo, restores any drifted frozen metadata, removes dev tooling + only the dev-link **symlinks** in `tests/xpacks` (never the installed deps), runs the gate ( `FAST=1` → cortex-m4f + native gcc14 + the two SMP platforms; otherwise the full 72 + `test-smp-all` ), and stops. It never pushes.

**Dry-run result (local, no push — confirmed `git ls-remote` shows 0 branches and 0 tags on every origin).** - **Clean reconcile:** merging `origin/xpack-development` into `step/30` in all three repos is *already-up-to-date* — the baseline is a clean ancestor, so the 30-step integration neither diverges from nor conflicts with `xpack-development`. - **Pristine held end-to-end:** on `step/30` vs baseline, `.github`, `README.md` and `LICENSE` are **byte-identical** in all three repos; the only touched existing test-framework files are `tests/package.json` and `tests/cmake/global-definitions.cmake`, both **semantically additive** (JSON subset check: every baseline entry preserved; global-definitions: 0 removed lines). Net additions over the whole integration: 5 build configurations ( `2xcortex-m33-*`, `native-smp-*`, `arm-cmsis-core-dependencies` ) and 3 actions ( `test-2xcortex-m33-cmake`, `test-native-smp`, `test-smp-all` ) — nothing modified or removed. - **Gate caveat:** `finalize.sh`'s "remove dev-link symlinks then run the gate" sequence assumes the

ports are **published** (consumed as released xpacks). In this local dry run the ports are dev-linked working copies (Step 24 cut their tags only locally), so the gate runs against those; the SMP legs are already proven green on them (m33 4/4, native-smp 3/3). The full 72 + `test-smp-all` with published ports is the CI acceptance run.

---

## 8. Quick Reference — Full Loop for Any Step

---

```
export WORK="$HOME/Documents/Work/micro-os-plus"
cd "$WORK/micro-os-plus-iii"

scripts/smp/new-step.sh 07 # branch(es) for step 7
$EDITOR src/rtos/internal/os-lists.cpp \
 src/rtos/os-timer.cpp \
 include/cmsis-plus/rtos/os-thread.h # apply $4 Step 7 edits
scripts/smp/verify-step.sh 07 # 72/72 gate (+unifdef if 14-23)
scripts/smp/advance-step.sh 07 "timer ISR re-entrancy"
scripts/smp/new-step.sh 08 # next
```

If `verify-step.sh` fails: read the failing config, fix, re-run verify. Never `advance` on red. All work stays local — no script pushes anything.

---

## 9. Step / Repo / Gate Matrix

Step	Part	Repos	Extra gate
01-12	A	K	72/72
13	A	K+P+C	72/72
14	B	K+P	unifdef + 72/72
15,16,18,19,21	B	K+P+C	unifdef + 72/72
17	B	K+P	unifdef + 72/72
20,22	B	K	unifdef + 72/72
23	B	K+P	unifdef + 72/72 + NCPUs=2 smoke
24,25	C	K+P+C	72/72
26,27,28	C	K(+tests)	72/72 + test-smp-all
29	D	docs	clean doc build
30	D	all	test-all + test-smp-all

## 10. End-to-End Command Flow (From an Empty Folder)

This section is the complete, copy-paste path: start in an empty directory, clone the repos with both remotes, then run the `new → edit → verify → advance` loop for every step until the upstream PR is ready. It assumes the helper scripts of §2 exist; §10.2 bootstraps them too, so a truly empty machine can start here.

### 10.1 Repos & branches

One repo per component, one `origin` remote each, on `github.com/dan-maio/`. The baseline and the source are **branches**, not separate repos:

Repo ( <code>github.com/dan-maio/...</code> )	<code>xpack-development</code> branch	<code>smp</code> branch
<code>micro-os-plus-iii</code>	single-core baseline (integrate into)	multi-core source
<code>micro-os-plus-iii-posix-arch</code>	baseline (tag <code>v1.0.1</code> )	source
<code>micro-os-plus-iii-cortexm</code>	baseline (tag <code>v1.1.0</code> )	source

Change `GH_ORG` in `scripts/smp/common.sh` if your repos live elsewhere. The diff a step lifts is simply `origin/xpack-development .. origin/smp`.

## 10.2 Bootstrap — clone everything into an empty folder

The real script is `../scripts/smp/bootstrap.sh`. It clones each repo once from `github.com/dan-maio`, makes both branches available, and installs xpm deps:

```
mkdir -p ~/smp-integration && cd ~/smp-integration
export WORK="$PWD"
bash /path/to/micro-os-plus-iii/scripts/smp/bootstrap.sh
```

What it does per repo: `git clone` → `git fetch --tags origin` → check out `xpack-development` → create a local tracking branch for `smp` → (kernel) `cd tests && xpm install`. Afterwards `common.sh` picks up `$WORK` and `$UPSTREAM = origin/xpack-development` is correct.

## 10.3 Extracting each step's chunk from `smp`

You do not hand-type the fixes — you lift them from the `smp` branch of the same repo. Two ways:

```
A) Review the baseline-vs-smp diff for a file, then apply the cohesive chunk:
scripts/smp/show-chunk.sh 07 src/rtos/internal/os-lists.cpp src/rtos/os-timer.cpp
(wraps: git diff origin/xpack-development origin/smp -- <file>)

B) If per-step commits were tagged smp-step-NN on the smp branch, apply directly:
git cherry-pick -n smp-step-07 # -n = stage only, so you can trim to scope
```

`run-loop.sh` with `AUTO=1` uses form B automatically when `smp-step-NN` exists.

**Saved recipes.** Each step's exact chunk — the file list plus any strip or surgical edit — is recorded as `../scripts/smp/chunks/stepNN.sh` (a reproducible record, verified byte-identical to the committed step). When a recipe exists, `run-loop.sh` applies it automatically instead of prompting, so a re-run of the whole series is deterministic. The recipe files also document *why* each multi-step file is stripped or surgically inserted (e.g. `step04.sh` strips the `destroying` assert; `step07.sh` inserts only the `errno` hunk; `step01.sh` keeps baseline `timegm.c`).

### Chunk curation — single-step vs. multi-step files

Two kinds of file, and they are lifted differently:

- **Single-step files** — the file's entire `xpack-development..smp` diff belongs to one step. Copy it wholesale: `git checkout origin/smp -- <file>`. Most files are like this (e.g. Step 3's `new.cpp`, Step 5's `file-descriptors-manager.cpp`).
- **Multi-step files** — the file is edited by several steps, so its full diff mixes concerns. A wholesale copy pulls in **future** content that either breaks the build (a symbol not introduced yet) or violates bisectability (Part B code in a Part A step). Here you lift **only the current step's hunks** and strip the rest.

Worked example (Step 4): `git checkout origin/smp -- src/rtos/os-c-wrapper.cpp` also brought in `static_assert(os_thread_state_destroying ...)` (Step 9 — and it *fails to compile*, that enum arrives in Step 9) and two `#if defined(OS_USE_SMP_SCHEDULER)` affinity blocks (Step 20). Step 4 strips all three, leaving only its own change (one-shot timer, polymorphic delete, 64-bit timeout).

**Detect a multi-step file before copying** — two cheap checks:

```
(1) how many step branches already touched it (does its history span steps)?
git log --oneline origin/xpack-development..origin/smp -- <file> | head

(2) does its smp diff carry content from a later part (SMP / new states)?
git diff origin/xpack-development origin/smp -- <file> \
 | grep -E '^+' | grep -icE 'OS_USE_SMP_SCHEDULER|_this_cpu|cpu_affinity|destroying|port_cpu_id'
```

A non-zero count on (2), or a `#if defined(OS_USE_SMP_SCHEDULER)` / a later-step symbol in the diff, means **trim, don't wholesale-copy**. Known multi-step hot files (verify before each step that touches them):

File	Touched by steps	Watch for
<code>src/rtos/os-c-wrapper.cpp</code>	4, 9, 20	<code>os_thread_state_destroying</code> , SMP affinity <code>#if</code>
<code>src/rtos/os-thread.cpp</code>	1, 9, 14, 20	exported-symbol change (1), destroying/ join (9), <code>port_cpu_id</code> /affinity (14,20)
<code>include/cmsis-plus/rtos/os-thread.h</code>	1, 7, 9, 14, 20	errno hunk (7), destroying enum (9), affinity/ <code>current_thread_</code> (14,20)
<code>include/cmsis-plus/rtos/os-c-decls.h</code>	9, 10, 16	<code>destroying = 7</code> (9), <code>os_condvar_t</code> <code>void* clock</code> (10), <code>smp_klock</code> (16)
<code>src/rtos/os-idle.cpp</code>	9	reaper/destroying
<code>src/rtos/os-core.cpp</code>	14, 15, 16, 18, 19, 22	almost entirely Part B — lift per step, all under <code>#if</code>

## Two lifting tools for entangled files

- `sc-project.py` (single-core projection) — strips every `#if defined(OS_USE_SMP_SCHEDULER)` and `OS_INTEGER_RTOS_PORT_NCPU > 1` block (dependency-free `unifdef` equivalent). Use it when a file is *heavily* SMP-gated (e.g. Step 9's `os-thread.cpp` has ~37 SMP hunks):  
`git show origin/smp:<f> | scripts/smp/sc-project.py > <f>`.
- **surgical insert / strip** with `perl` — for a single small hunk.

**Projection caveat (learned the hard way at Step 9).** The single-core projection keeps **all** of a file's *non-SMP* changes at once — it cannot tell Step 9's changes from Step 10's. It is exact **only when every non-SMP Part-A step that touches the file is ≤ the current step**. `os-c-decls.h` is

touched by Step 9 ( `destroying` ) and Step 10 ( `os_condvar_t` 's `void* clock` ); projecting it at Step 9 pulled the Step-10 field in, growing `os_condvar_t` to 16 bytes while `condition_variable` stayed 12 → `static_assert (sizeof ... == ...)` failed. The size assert caught it, but the lesson stands: **before projecting a file, check it has no later non-SMP step** ( `git log origin/xpack-development..origin/smp -- <f>` and scan the diff). If it does, project the safe files and apply the current step's change to the entangled one surgically (Step 9 projects `os-thread.{h,cpp}` + `os-idle.cpp` , but inserts only the `destroying` enumerator into `os-c-decls.h` ).

For Part B multi-step files the `unifdef` invariant (Stage 1 of `verify-step.sh` ) is the backstop: if a lifted hunk leaks outside `#if defined(OS_USE_SMP_SCHEDULER)` it fails there before the build. ( `sc-project.py` can stand in for `unifdef` there too, avoiding that dependency.)

## 10.4 The main loop — Steps 1 through 23 (K + optional P/C)

The loop is `../scripts/smp/run-loop.sh` :

```
export WORK="$HOME/smp-integration"
cd "$WORK/micro-os-plus-iii"
scripts/smp/check-env.sh

scripts/smp/run-loop.sh 1 23 # interactive: pauses per step to apply the chunk
or, if smp-step-NN commits are tagged on the smp branch:
AUTO=1 scripts/smp/run-loop.sh 1 23 # unattended: cherry-picks each chunk
```

Each iteration runs `new-step.sh NN` → apply chunk (the one manual action, unless `AUTO=1` ) → `verify-step.sh NN` (unifdef for 14–23 + link + 72 tests) → `advance-step.sh NN` (commit + tag `step-NN-green` ), then loops to NN+1 which forks from the just-tagged green step. Any red step stops the loop.

## 10.5 Part C onward — Steps 24 through 30

Part C introduces the new repos and add-only tests; run these after 23 is green.

```

Clone the remaining port/arch repos now that Part C needs them.
for r in aarch32 aarch64 devices riscv; do
 git clone https://github.com/dan-maio/micro-os-plus-iii-$r.git \
 "$WORK/micro-os-plus-iii-$r" 2>/dev/null || true
done

scripts/smp/run-loop.sh 24 28 # add-only Part C ($6)

Step 29 – docs
scripts/smp/new-step.sh 29
cd docs && ./render-pdfs.sh Implementation-SMP-Integration && cd ..
scripts/smp/advance-step.sh 29 "docs + PDFs"

Step 30 – final merge, drop dev tooling, run both gates (all local)
scripts/smp/new-step.sh 30
scripts/smp/finalize.sh
finalize.sh ends green on the local step/30 branch. Publishing is manual and
is NOT part of any script – when you decide to, push and open the PR by hand:
git -C "$WORK/micro-os-plus-iii" push origin step/30
gh pr create --repo dan-maio/micro-os-plus-iii \
--base xpack-development --head step/30 \
--title "SMP multi-core integration (30-step bisectable series)" \
--body-file docs/smp-integration/Implementation-SMP-Integration.md

```

## 10.6 One-glance flow

```

empty folder
| bootstrap.sh clone upstream+smp, wire remotes, xpm install
▼
xpack-development baseline
| ┌────────────────────────── loop NN = 01 ... 30 ───────────────────┐
| | new-step.sh NN fork step/NN from step/NN-1 (K [+P +C]) |
| | show-chunk.sh NN ... review smp diff → apply cohesive chunk |
| | verify-step.sh NN unifdef(14-23) + link 24 configs + 72 tests |
| | advance-step.sh NN commit + tag step-NN-green → base for NN+1 |
| └──────────────────────────┘
▼
finalize.sh (local: merge + both gates) — manual, by hand —► push + open PR

```

## 11. Doing This For Real — The Practical Playbook

This chapter is the plain-English version of everything above: what you actually do, in what order, what the scripts do for you, what only a human can do, and — above all — why the whole thing is built to be **repeatable**. If you read only one section before starting, read this one.

### 11.1 The one-sentence mental model

You are **not** merging the `smp` branch into `xpack-development`. You are **rebuilding** the SMP work on top of `xpack-development`, one small, reviewed, independently-tested piece at a time, so that the result is a clean, bisectable series of commits that upstream can actually review — instead of one giant "here are 10,000 changed lines" merge that nobody can check.

Think of it as two Git repositories playing two fixed roles:

- **`smp` branch — the source of truth.** It already works (its own tests pass). You never change it. You only *read* it: "what did the SMP version do to this file?"
- **`xpack-development` branch — the destination.** This is upstream's current single-core code. It is what you integrate *into*, and what must stay buildable and test-green after **every** step.

Every step is the same move: **lift one cohesive idea** out of `smp`, drop it onto `xpack-development`, prove the result still builds and passes, commit, tag, repeat.

### 11.2 Why repeatability is the whole point

A one-shot merge is a dead end: if it breaks, you cannot tell *which* of the thousands of changes broke it, and a reviewer cannot sign off on it. This runbook is engineered so the entire integration can be **thrown away and regenerated from scratch, deterministically**, as many times as needed. Concretely, repeatability means:

1. **Every edit is a recipe, not a keystroke.** The per-step chunk scripts in `scripts/smp/chunks/` reproduce each change *byte-for-byte* from `origin/smp` and the baseline. Nobody hand-edits files. Re-running a chunk on a fresh clone produces exactly the same bytes — verified by diffing against the committed step.
2. **Every edit fails loudly if the source moved.** Surgical edits go through `_edit.py`, which **asserts** its anchor text is present the exact expected number of times. If a future `smp` revision renames a function, the recipe stops with an error instead of silently doing nothing. (This is why we replaced the original `perl` one-liners — a `perl s///` that matches nothing just succeeds, and a silent miss is how a `-Werror` regression sneaks in.)
3. **Every step is a labelled checkpoint.** `advance-step.sh` tags each green step `step-NN-green`. The next step forks from it. You can `git checkout step-17-green` at any time and be exactly where that step left off. Bisecting a later failure is trivial.
4. **The test framework is frozen, and that is enforced, not trusted.** `check-pristine.sh` runs before every build and every commit and refuses any change that modifies or deletes an existing test,

platform, CI file, or config entry (Parts A/B are totally frozen; Part C allows *additions only*, checked by a semantic JSON-subset comparison, not a fragile line diff).

5. **Nothing leaves your machine until you say so.** No script pushes, opens a PR, or writes to any remote. The entire 30-step series lives on local branches and tags; publishing is a deliberate, manual `git push` you run by hand at the end.

If any of these five is violated, the series is no longer trustworthy — so the scripts treat each one as a hard gate, not a nicety.

## 11.3 What you do, concretely, from an empty folder

The fully-scripted path is in §10; here it is in words.

1. **Set up once.** Install the known-good Node/xpm (§1), then run `scripts/smp/bootstrap.sh`. It clones the repos, checks out the `xpack-development` baseline, and does the first `xpm install`. You now have a buildable single-core baseline and the `smp` branch sitting beside it to read from.
2. **Do Part 0 first** (`migrate-devices.sh`): dissolve the `devices` repo into the architecture repos, because the SMP ports assume that layout. Skipping this is the most common way to get stuck later (the port CMake will ask for a `devices` sibling that should no longer exist).
3. **Loop, one step at a time.** For step `NN`:
4. `new-step.sh NN` — forks `step/NN` from the previous green step.
5. **Read the `smp` diff for this step's files** (`show-chunk.sh NN <file>`), and apply the chunk recipe. *This is where your judgement lives* — see 11.4.
6. `verify-step.sh NN` — links the local ports into the test configs and runs the gate (FAST subset during development; the full 72 for a real acceptance).
7. `advance-step.sh NN "message"` — runs the pristine check, commits each touched repo, tags `step-NN-green`.
8. Only move to `NN+1` when `NN` is green. Never carry a red step forward.
9. **Finish** with `finalize.sh` (Part D): it reconciles with the latest baseline, drops the dev-only tooling, and runs the gate one last time — all locally.
10. **Publish by hand** if and when you choose. That is the only non-scripted step, on purpose.

## 11.4 The parts a script cannot do for you

Automation reproduces decisions; it does not *make* them. Budget real time for:

- **Deciding each chunk's boundary.** A "step" is one cohesive idea, which may span several files across several repos (a kernel declaration *and* the port definition that satisfies it must travel together, or the build breaks). Reading the `smp` diff and grouping it into the smallest self-contained, test-green unit is human work. The existing `step01`–`step13` recipes are worked examples.
- **Choosing the recipe kind.** Whole-file copy when `smp`'s version of a file is a clean single-step change; *single-core projection* (`sc-project.py`) when a file is riddled with `#if OS_USE_SMP_SCHEDULER` blocks; *surgical insert* when a file mixes several steps' content and only one

hunk belongs now. Picking wrong usually shows up as a later step having nothing left to do, or an early step failing to build.

- **Handling what the real build finds.** The SMP source was proven in its *own* standalone build, which differs from the xpack-development harness (stricter `-Werror`, different libc, host vs. cross compiler). Expect genuine port engineering — our dry run surfaced real ones: `struct timespec` vs `timespec` under `-Werror=redundant-tags`; a `uc_sigmask` field that exists only in the system `ucontext`, not the xPack one; and a kernel/port double-declaration of `port_cpu_id` that *only appears under a true SMP compile*. When you hit one, fix it in the recipe (so it reproduces), not just in the working tree.
- **Knowing when "green" is real.** The sharpest lesson of the dry run: defining `OS_USE_SMP_SCHEDULER` on the `cmake / xpm` command line as a `-D` **cache variable** does *not* make it a compile flag, so the code quietly builds single-core and the "SMP" tests pass meaninglessly. SMP is only truly on when the define comes from a platform's `target_compile_definitions`. Verify the flag is on the actual compile line (`cmake --build <dir> -v`), not just in `CMakeCache`.

## 11.5 Supplementary actions worth building into your routine

- **Re-run from scratch periodically.** The point of deterministic recipes is that you can delete the whole scratch workspace and regenerate the series. Do it at least once before publishing, on a clean checkout, to prove the recipes don't secretly depend on leftover state.
- **Keep the PDF in step with the Markdown.** `python3 docs/md2pdf.py docs/Implementation-SMP-Integration.md docs/Implementation-SMP-Integration.pdf` after any edit (or `docs/render-pdfs.sh`).
- **Pin your toolchains.** Use the xPack-pinned GCC/clang/arm-none-eabi/QEMU, never the machine's system compiler (it is typically too new — the host GCC emitted assembler directives its own `as` rejected during the dry run). The test configs already name exact versions.
- **Record the environment gotchas you hit.** On this machine, `xpm 0.23.3` on Node 26 intermittently created a toolchain's `.bin` shims without linking the package folder; the fix was to link the pinned toolchain folder by hand and put its `bin` on `PATH`. Your environment will have its own; note them so the next run is faster.
- **VS Code xPack Extension & `link-deps`:** When preparing or building targets via VS Code's xPack extension tree, `package.json` must specify `link-deps` and include `xpm link @micro-os-plus/micro-os-plus-iii-<port> --config {{ configuration.name }}` inside `actions.install`. Without this, running `install` in VS Code only downloads toolchains without creating the dev-linked port symlinks in `build/<config>/xpacks/`, causing `prepare` to fail with `Missing .../CMakeLists.txt`.
- **CMake Symlink REALPATH Resolution & Target Guards:** When ports are dev-linked via `xpm link`, `${CMAKE_CURRENT_SOURCE_DIR}` is a symlink inside the build directory. Using `get_filename_component(_uos_real_source "${CMAKE_CURRENT_SOURCE_DIR}" REALPATH)` ensures child ports locate true sibling repositories (`micro-os-plus-iii-devices`, `micro-os-plus-iii`). The kernel `CMakeLists.txt` guards against duplicate aliases with `if (TARGET micro-os-plus::iii) return() endif()` and provides the `micro-os-plus::iii-core` interface target alias.

- **Treat every new finding as a recipe change.** If you fix something in a file, put the fix in the chunk script and re-verify it reproduces from pristine `origin/smp`. A fix that lives only in your working tree is a fix you will lose on the next clean run.

## 11.6 The repeatability checklist (run before you publish)

- [ ] Fresh clone + `bootstrap.sh` + the full loop reproduces every `step-NN-green` byte-for-byte.
- [ ] `check-pristine.sh` passes at every step (frozen in A/B, additive-only in C).
- [ ] The single-core projection of every Part B step is identical to the previous step (no SMP code leaked outside its `#if`).
- [ ] The acceptance gate is green with SMP defines coming from the **platform**, not the command line.
- [ ] `git ls-remote` shows **zero** of your `step/*` branches and `step-*-green` tags on any origin — nothing was pushed.
- [ ] The final `step/30` merges the latest `xpack-development` with no conflicts, and `.github` / `README` / `LICENSE` are byte-identical to baseline.

Only when all six hold is the series ready for you to push and open the PR — by hand.